Web and Data Engineering

Lecture 04: HTTP und RESTful APIs

Prof. Dr. Michael Granitzer



Speaker notes

Diese Vorlesung behandelt HTTP als Kommunikationsprotokoll des Webs und REST als darauf aufbauenden Architekturstil für APIs (Application Programming Interfaces, also Programmierschnittstellen). Im Mittelpunkt stehen Nachrichtenaufbau, HTTP-Semantik, Caching, Sicherheit, Ressourcenmodellierung und der Lebenszyklus von APIs.

Ziel dieser Vorlesung

HTTP ist das Protokoll des Webs. REST nutzt dieses Protokoll konsequent als Grundlage für API-Design.

> Diese Vorlesung verbindet beides: vom **Protokollverständnis** über **Caching und Sicherheit** bis zum **Entwurf robuster, evolvierbarer APIs**.

Leitfragen

- Warum ist HTTP mehr als Datentransport?
- Wie strukturieren Header HTTP-Nachrichten und ihre Repräsentationen?
- Wie steuern Header Caching, Sessions und Sicherheit?
- Was unterscheidet REST von beliebigem JSON-über-HTTP?
- Wie modelliert man Ressourcen, URIs und HTTP-Semantik für APIs?
- Wie entwickelt man APIs weiter, ohne Clients zu brechen?

Speaker notes

HTTP definiert, wie Clients und Server Nachrichten austauschen. REST nutzt diese Mechanismen konsequent für API-Design. Lernziel ist, HTTP-Nachrichten fachlich zu lesen, Header, Medienformate und Statuscodes korrekt zu interpretieren und APIs (Application Programming Interfaces) anhand von Ressourcen, Semantik und Evolvierbarkeit zu bewerten.



HTTP Grundlagen

Leitfrage: Wie können Browser und Server mit einem einfachen, textbasierten Protokoll so kommunizieren, dass das Web weltweit skalierbar bleibt?

Speaker notes

HTTP ist ein standardisiertes Request-Response-Protokoll. Es trennt klar zwischen Client und Server und bildet die Grundlage für Web-Seiten, Web-APIs, Caching-Infrastruktur und viele Sicherheitsmechanismen.

Was sehen Sie hier?

Ein Nutzer ruft die Produktseite auf. Das Netzwerk-Tab zeigt diesen Austausch:

↑ REQUEST

```
GET /api/products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
Cookie: session=abc123
```

↓ RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300
ETag: "f7e3a1b2"

{"id":42,"name":"Funkkopfhörer","price":79.99}
```

Aufgabe: Was ist Startzeile, Header, Body? Welche Information steckt in jedem Header?

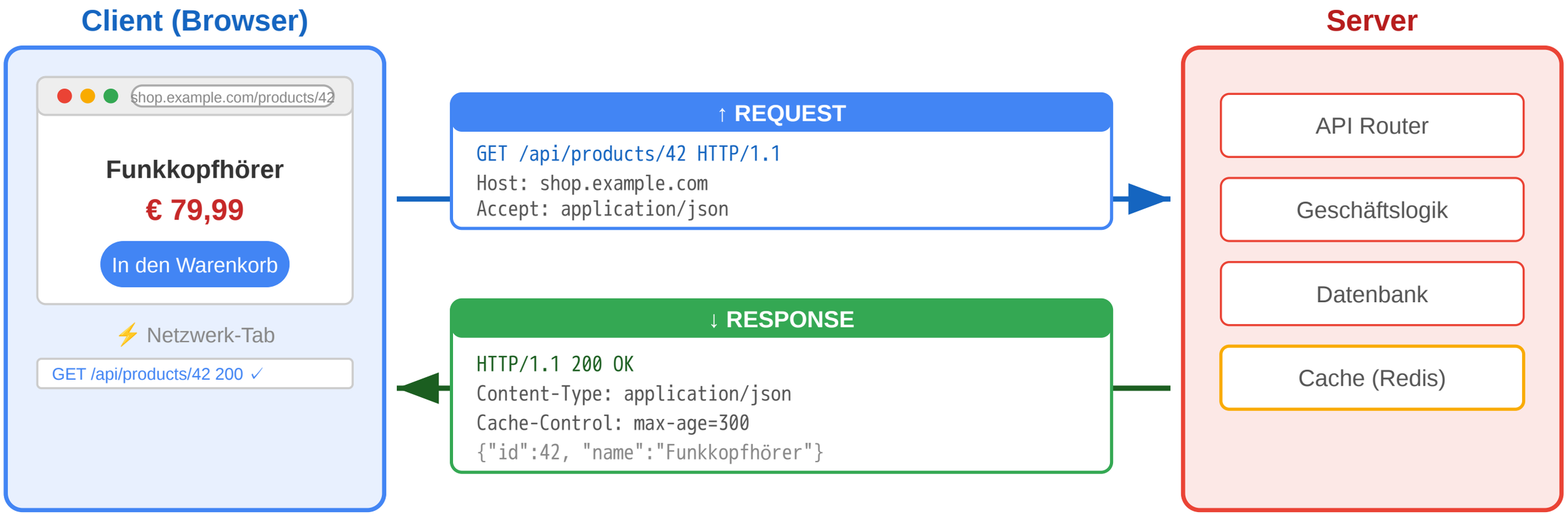
Teil	Request	Response
Startzeile	`GET /api/products/42 HTTP/1.1`	`HTTP/1.1 200 OK`
Header	Host, Accept, Cookie	Content-Type, Cache-Control, ETag
Body	kein Body bei GET	JSON-Daten des Produkts

Cache-Control: max-age=300
Daten 5 min gültig
ETag: "f7e3a1b2"
Fingerprint zur Validierung

Speaker notes

Ein HTTP-Austausch besteht aus Startzeile, Headern und optionalem Body. Im Request beschreibt die Startzeile Methode, Ziel und Protokollversion. Im Response beschreibt die Startzeile Version, Statuscode und Statusmeldung. Header transportieren Metadaten wie akzeptierte Formate, Session-Informationen oder Cache-Regeln. Der Body enthält die eigentliche Repräsentation der Ressource, hier JSON (JavaScript Object Notation).

HTTP: Das Request-Response-Protokoll



Dazwischen: DNS · CDN/WAF · Load Balancer

Speaker notes

HTTP arbeitet nach einem einfachen Muster: Der Client sendet einen Request an eine Adresse, der Server verarbeitet ihn und liefert einen Response. Diese Struktur ermöglicht eine lose Kopplung zwischen Browsern, Servern, Proxies, Caches und Gateways.

Von der Fetch API zum Protokoll

In Vorlesung 03 haben wir die **Client-Seite** gesehen:

```
const response = await fetch('/api/products/42', {
  headers: { 'Accept': 'application/json' }
});
```

Jetzt schauen wir auf die **Protokollseite**:

Fetch API	HTTP-Protokoll
`fetch(url)`	Browser erzeugt HTTP-Request mit Startzeile + Headern
`{ method: 'POST' }`	HTTP-Methode — definiert Absicht und Semantik
`{ headers: {...} }`	Request-Header — Metadaten für Server und Infrastruktur
`response.status`	Statuscode — standardisiertes Ergebnis
`response.json()`	Response Body parsen — Repräsentation der Ressource

> Die Fetch API ist eine **JavaScript-Abstraktion über HTTP**. Wer HTTP versteht, versteht auch, warum `fetch` so funktioniert wie es funktioniert — und kann Netzwerkprobleme im DevTools-Netzwerk-Tab diagnostizieren.

Speaker notes

Die Fetch API ist eine Programmierschnittstelle im Browser, die HTTP-Nachrichten erzeugt und Responses kapselt. `method`, `headers` und Body in JavaScript entsprechen direkt Bestandteilen einer HTTP-Nachricht. Wer HTTP versteht, kann Netzwerkverhalten unabhängig vom verwendeten Framework analysieren.

HTTP-Designprinzipien

Prinzip	Bedeutung	Konsequenz
Ressourcenorientiert	Jede Information hat eine Adresse (URI)	Einheitliche Adressierung weltweit
Request-Response	Client fragt, Server antwortet	Klare Rollenverteilung
Zustandslos	Jeder Request ist unabhängig	Server muss keinen Kontext halten → skaliert
Erweiterbar	Header erlauben beliebige Metadaten	Caching, Auth, Content-Negotiation ohne Protokolländerung
Textbasiert	Nachrichten sind menschenlesbar (HTTP/1.1)	Einfach zu debuggen

Warum das skaliert

- Zustandslosigkeit erlaubt Load Balancing über beliebig viele Server
- Proxies und Caches können HTTP-Nachrichten ohne Anwendungswissen verarbeiten
- Klare Trennung von Transport und Anwendungslogik

Speaker notes

HTTP ist ressourcenorientiert, zustandslos und durch Header erweiterbar. Zustandslosigkeit bedeutet, dass jeder Request für sich interpretierbar sein muss. Dadurch werden Lastverteilung, Zwischenspeicherung und horizontale Skalierung erleichtert. URIs (Uniform Resource Identifiers) geben Ressourcen stabile Adressen.

URI (Uniform Resource Identifier)

Schema

`scheme://host:port/path?query#fragment`

Example

`https://shop.example.com:443/api/products/42?lang=de#details`

Teil	Beispiel	Bedeutung
Scheme	`https`	URI-Schema vor `:`; bestimmt, wie die URI interpretiert wird
Host	`shop.example.com`	DNS-Name des Servers
Port	`443`	TCP-Port; bei HTTPS implizit 443
Pfad	`/api/products/42`	Adresse der Ressource auf dem Host
Query	`?lang=de`	Parameter für Auswahl, Filter, Darstellung
Fragment	`#details`	clientseitiger Sprunganker; wird nicht an den Server gesendet

HTTP-Methoden als semantischer Vertrag

Methode	Absicht	Sicher?	Idempotent?	Online-Shop-Beispiel
GET	Ressource lesen	ja	ja	Produktseite anzeigen
POST	Neue Ressource oder Verarbeitung	nein	nein	Bestellung aufgeben
PUT	Ressource vollständig ersetzen	nein	ja	Profil aktualisieren
PATCH	Ressource teilweise ändern	nein	nein*	Adresse ändern
DELETE	Ressource entfernen	nein	ja	Konto löschen
HEAD	Nur Header, kein Body	ja	ja	Existenzprüfung
OPTIONS	Erlaubte Methoden abfragen	ja	ja	CORS Preflight

Sicher (Safe): Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.

Idempotent: Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

Achtung: Die Unterscheidung ist eine **Konvention**, keine technische Durchsetzung. Jeder Server kann jede Methode für jede semantische Bedeutung nutzen. Die Tabelle beschreibt die **übliche** Interpretation.

Speaker notes

HTTP-Methoden kodieren Absichten. `GET` liest, `POST` erzeugt oder stößt Verarbeitung an, `PUT` ersetzt, `PATCH` ändert teilweise, `DELETE` entfernt. Safe bedeutet: keine Änderung am fachlichen Zustand. Idempotent bedeutet: Mehrfachausführung hat denselben Effekt wie einmalige Ausfuehrung.

HTTP-Methoden: Safe und Idempotent in der Praxis

Szenario: Ein Nutzer klickt zweimal schnell auf „Jetzt kaufen“. Netzwerk-Lag verzögert den ersten Request, beide kommen beim Server an.

Methode	Effekt beim Doppel-Request	Warum?
`POST /api/orders`	**Zwei Bestellungen** entstehen	Nicht idempotent — jeder Aufruf erzeugt neue Ressource
`PUT /api/cart/item/42`	**Eine Änderung** — zweiter Call ohne Effekt	Idempotent — gleiches Ergebnis egal wie oft

Konsequenz: Load Balancer und API-Gateways können idempotente Requests bei Timeout sicher wiederholen. CDNs cachen nur **safe** Methoden (GET, HEAD), weil diese die Ressource nie verändern.

Speaker notes

Die Unterscheidung zwischen safe und idempotent hat direkte Auswirkungen auf Betrieb und Fehlertoleranz. Idempotente Requests können bei Netzwerkproblemen sicher wiederholt werden. Nicht-idempotente POST-Requests brauchen zusätzliche Schutzmechanismen, damit Wiederholungen keine doppelten Effekte erzeugen.

HTTP-Statuscodes

Klasse	Bedeutung	Wichtige Codes
1xx	Information	`101 Switching Protocols`
2xx	Erfolg	`200 OK`, `201 Created`, `204 No Content`
3xx	Umleitung	`301 Moved Permanently`, `304 Not Modified`
4xx	Client-Fehler	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `429 Too Many Requests`
5xx	Server-Fehler	`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

Was stimmt an dieser API-Antwort nicht?

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "error": "Produkt nicht gefunden"
}
```

Problem: Statuscode lügt

- `200 OK` signalisiert Erfolg
- Body enthält aber einen Fehler
- CDNs cachen die „Erfolg“-Antwort
- Monitoring zählt 200 als OK

Richtig: `404 Not Found` als Statuscode, Fehlerdetails im Body.

Speaker notes

Statuscodes sind maschinenlesbare Standardantworten. 2xx steht für Erfolg, 3xx für Umleitungen, 4xx für Probleme auf Client-Seite und 5xx für Probleme auf Server-Seite. Der Statuscode beschreibt die eigentliche Bedeutung des Ergebnisses; ein Fehler darf nicht nur im Body versteckt werden.

HTTP-Versionen im Vergleich

Version	Jahr	Kernverbesserung
HTTP/1.0	1996	Ein Request pro TCP-Verbindung
HTTP/1.1	1997	Persistent Connections, Pipelining
HTTP/2	2015	Multiplexing, Header-Kompression, binär
HTTP/3	2022	QUIC (UDP-basiert), schnellerer Aufbau

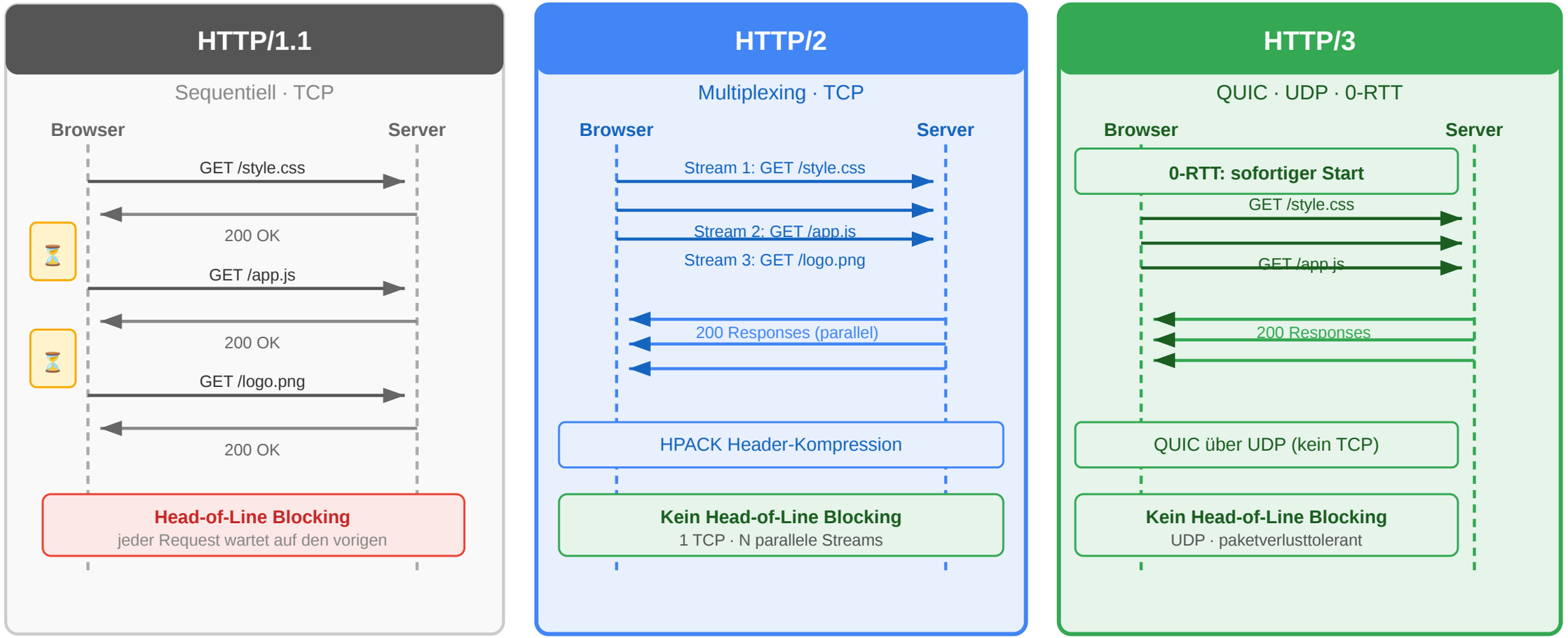
QUIC ist der Transport unter HTTP/3: Es läuft über UDP, integriert Verschlüsselung direkt und kann Verbindung plus TLS schneller aufbauen. Wichtig: QUIC ersetzt hier TCP, HTTP-Semantik wie Methoden, Statuscodes und Header bleibt erhalten.

HTTP ist nicht nur Datentransport — es ist das **Interaktionsmodell** des Webs. Methoden definieren Absichten, Statuscodes kommunizieren Ergebnisse, Header steuern Verhalten. Infrastruktur (CDN, Load Balancer, WAF) verlässt sich auf diese Semantik. Wer HTTP als Syntax behandelt, verliert die Vorteile des Protokolls.

Speaker notes

HTTP/1.0 öffnete typischerweise pro Request eine neue TCP-Verbindung. HTTP/1.1 führte persistent connections ein. HTTP/2 multiplexiert mehrere Requests über eine Verbindung und komprimiert Header. HTTP/3 basiert auf QUIC (Quick UDP Internet Connections) über UDP (User Datagram Protocol) und reduziert Verbindungsaufbau und Transportlatenz.

HTTP-Versionen im Vergleich



Frage: Eine Produktseite lädt 60 Ressourcen (JS, CSS, Bilder). Warum dauert das mit HTTP/1.1 deutlich länger als mit HTTP/2 — obwohl die Bandbreite dieselbe ist?

HTTP Headers and Status Codes

Leitfrage: Wie strukturieren Header und Statuscodes HTTP-Nachrichten, sodass Browser, Caches und Anwendungen ohne fachliches Vorwissen entscheiden können, was zu tun ist?

Speaker notes

Header und Statuscodes sind die zwei semantischen Achsen einer HTTP-Nachricht. Header tragen Metadaten — was wird übertragen, in welchem Format, mit welcher Berechtigung, in welchem Caching-Verhalten. Statuscodes kommunizieren das Ergebnis — Erfolg, Weiterleitung, Client- oder Server-Fehler. Beide sind maschinenlesbar und Infrastruktur (Browser, Caches, Load Balancer, Monitoring) reagiert ausschließlich auf sie, nicht auf Body-Inhalte. Wer diese Semantik korrekt nutzt, gewinnt Cachebarkeit, Beobachtbarkeit und Wiederverwendbarkeit fast geschenkt.

HTTP-Header im Überblick

Eine HTTP-Nachricht besteht aus:

1. **Startzeile**
   Request: Methode + Ziel + Version
   Response: Version + Statuscode + Statusmeldung
2. **Header-Felder**
   Metadaten als `Name: Wert`
3. **Leerzeile**
   Trennt Header und Body
4. **Optionaler Body**
   Die eigentliche Repräsentation

```
GET /products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
```

Speaker notes

Header-Felder stehen zwischen Startzeile und Body. Sie beschreiben nicht die Ressource selbst, sondern Zusatzinformationen zur Nachricht, etwa Zielhost, gewünschtes Format, Authentifizierung, Cache-Verhalten oder Cookie-Daten. Die Leerzeile ist technisch relevant, weil sie den Headerblock vom optionalen Body trennt. Erst durch diese Struktur können generische Werkzeuge wie Browser, Proxies oder Debugger Nachrichten systematisch lesen.

Wie sind HTTP-Header strukturiert?

Teil	Bedeutung	Beispiel
Feldname	Standardisierter Header-Name	`Content-Type`
Doppelpunkt	Trennt Name und Wert	`:`
Feldwert	Semantik des Headers	`application/json`

```
Content-Type: application/json; charset=utf-8
Cache-Control: public, max-age=3600
Authorization: Bearer eyJhbGciOi...
```

- Header-Namen sind **case-insensitive**
- Ein Feldwert kann aus **Token, Parametern oder Listen** bestehen
- Viele Header haben eine klar definierte **Syntax nach RFC**

Speaker notes

Die allgemeine Form ist immer `Feldname: Wert`. Die genaue Struktur des Werts hängt vom Header ab: Media Types haben optionale Parameter wie `charset`, Cache-Control kombiniert mehrere Direktiven, Accept-Header können mehrere Werte mit Prioritaeten enthalten.

Kategorien von Header-Feldern

Kategorie	Typische Header	Zweck
Request-Header	`Host`, `Accept`, `Authorization`, `Cookie`	Beschreiben den eingehenden Request
Response-Header	`Server`, `Location`, `Set-Cookie`	Beschreiben Eigenschaften der Antwort
Repräsentations-Header	`Content-Type`, `Content-Length`, `Content-Encoding`, `Content-Language`	Beschreiben den Body
Caching / Conditional	`Cache-Control`, `ETag`, `If-None-Match`, `Last-Modified`	Steuern Wiederverwendung und Validierung
Sicherheits-Header	`Strict-Transport-Security`, `Content-Security-Policy`	Erhöhen Sicherheitseigenschaften

Wichtig: Dieselbe HTTP-Ressource kann mehrere Repräsentationen haben. Die Header beschreiben, **welche Variante gerade übertragen wird**.

Speaker notes

Diese Kategorien helfen beim Lesen realer HTTP-Nachrichten. Besonders wichtig für das Verständnis von REST ist die Trennung zwischen Ressourcenidentität durch URI (Uniform Resource Identifier) und Repräsentationsbeschreibung durch Header.

Typische Request- und Response-Header

Header	Typ	Kurz erklärt
`Host`	Request	Gibt an, für welchen Host der Request bestimmt ist; wichtig bei virtuellen Hosts
`Authorization`	Request	Übertraegt Anmeldedaten oder Zugriffstoken
`Location`	Response	Verweist auf die URI einer neuen oder anderen Ressource
`Server`	Response	Kann Informationen über die Server-Software enthalten

```
POST /login HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJ...
```

Speaker notes

`Host` ist seit HTTP/1.1 zentral, weil mehrere Websites auf derselben IP-Adresse betrieben werden können. `Authorization` überträgt Zugriffsinformationen, etwa ein Bearer-Token. `Location` wird oft bei Redirects oder `201 Created` verwendet. `Server` ist für Clients meist nicht funktional nötig, taucht aber in realen Responses häufig auf.

Typische Header für Repräsentation und Kontrolle

Header	Kategorie	Kurz erklärt
`Content-Length`	Repräsentation	Nennt die Länge des Bodys in Bytes
`Content-Encoding`	Repräsentation	Gibt an, ob der Body z. B. mit gzip komprimiert wurde
`Content-Security-Policy`	Sicherheit	Beschränkt, welche Inhalte eine Seite laden oder ausführen darf
`Strict-Transport-Security`	Sicherheit	Erzwingt für eine Domain die Nutzung von HTTPS

Merksatz:
Ein Teil der Header beschreibt den **Body**, ein anderer Teil steuert **Verhalten** von Clients, Caches und Browsern.

Speaker notes

`Content-Length` und `Content-Encoding` beschreiben Eigenschaften der übertragenen Repräsentation, die im Alltag leicht übersehen werden. `Content-Security-Policy` und `Strict-Transport-Security` sind typische Sicherheits-Header für Web-Anwendungen; letzterer wird oft als HSTS für HTTP Strict Transport Security abgekuerzt.

Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

> **Aufgabe:** Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Speaker notes

Beim erfolgreichen Erzeugen einer Ressource ist `201 Created` der passende Statuscode. Ein `Location`-Header verweist auf die neue Ressource. Zusätzlich sollte klar sein, ob eine solche Response gespeichert werden darf; für Bestellungen ist `no-store` oft sinnvoll.

Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

> **Aufgabe:** Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Problem	Warum	Besser
`200 OK` statt `201 Created`	`200` sagt „gelesen", nicht „erzeugt"	`201 Created`
Kein `Location`-Header	Client weiß nicht, wo die neue Ressource liegt	`Location: /api/orders/789`
Kein `Cache-Control`	Bestellungen dürfen nicht gecacht werden	`Cache-Control: no-store`

Korrigierte Version

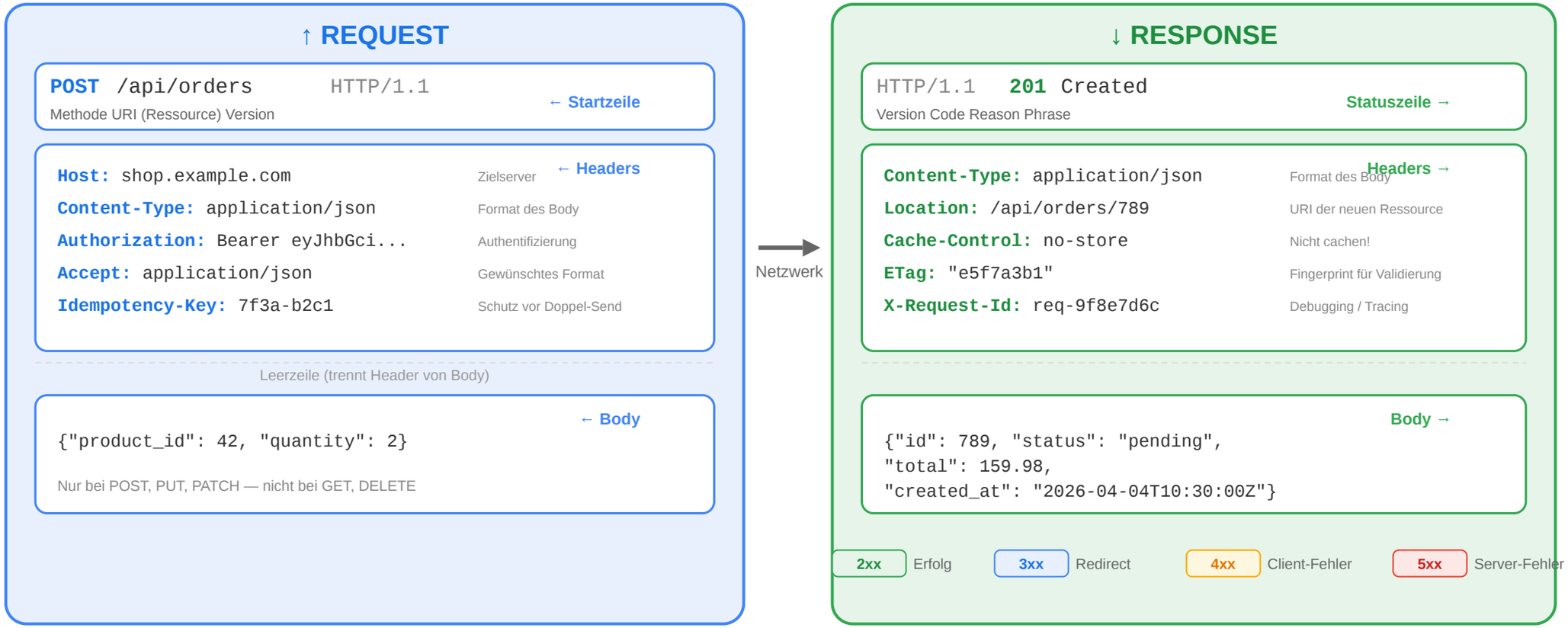


```
HTTP/1.1 201 Created
Location: /api/orders/789
Content-Type: application/json
Cache-Control: no-store
```

```
{"id": 789, "status": "pending", "total": 159.98}
```

Der vollständige Request-Response-Zyklus

Anatomie: HTTP Request und Response



Speaker notes

Der Zyklus umfasst Startzeilen, Header und optionale Bodies auf beiden Seiten. Requests enthalten typischerweise Ziel, Semantik und Metadaten; Responses enthalten Ergebnis, Beschreibungsdaten und die eigentliche Repräsentation.

Statuscodes richtig einsetzen

Situation	Richtiger Code	Häufiger Fehler
Ressource erfolgreich gelesen	`200 OK`	—
Ressource erfolgreich angelegt	`201 Created` + Location	`200` ohne Location
Erfolgreich, aber kein Body	`204 No Content`	`200` mit leerem Body
Ungültige Eingabedaten	`400 Bad Request`	`500` (ist kein Server-Fehler!)
Nicht authentifiziert	`401 Unauthorized`	`403`
Authentifiziert, aber nicht berechtigt	`403 Forbidden`	`401`
Ressource nicht gefunden	`404 Not Found`	`200` mit Fehlermeldung im Body
Konflikt (z.B. doppelter Username)	`409 Conflict`	`400`

Faustregel:
Der Statuscode sagt dem Client, **was passiert ist**. Der Body erklärt **warum** und **was zu tun ist**.

Speaker notes

Die Semantik der 4xx-Codes ist präziser als oft angenommen: `401 Unauthorized` bedeutet, dass die Identität fehlt oder ungültig ist — der Client soll sich neu authentifizieren. `403 Forbidden` bedeutet, dass die Identität bekannt, aber die Berechtigung nicht vorhanden ist — erneutes Einloggen hilft nicht. `400 Bad Request` beschreibt syntaktisch oder strukturell ungültige Eingaben, `422 Unprocessable Entity` fachlich ungültige (aber korrekt formatierte) Eingaben, `409 Conflict` fachliche Konflikte (z.B. Duplikat). `204 No Content` signalisiert Erfolg ohne Rückgabe-Body — für `DELETE` und `PUT` gebräuchlich. Der Statuscode ist der maschinenlesbare Kern der Antwort: Caches, Load Balancer und Monitoring-Systeme reagieren auf Codes, nicht auf Body-Inhalte. Falsch gewählte Codes (z.B. `200` mit Fehlerbeschreibung im Body) machen die API für automatisierte Verarbeitung unbrauchbar.

Fehlermodelle: Konsistenz ist alles

```
{
  "error": {
    "type": "validation_error",
    "message": "Die Eingabedaten sind ungültig.",
    "details": [
      { "field": "email", "code": "invalid_format",
        "message": "Keine gültige E-Mail-Adresse" },
      { "field": "quantity", "code": "out_of_range",
        "message": "Muss zwischen 1 und 100 liegen" }
    ]
  }
}
```

Prinzip	Umsetzung
Konsistente Struktur	Immer dasselbe JSON-Format für Fehler
Maschinenlesbar	`type` und `code` für automatische Verarbeitung
Menschenlesbar	`message` für Entwickler und Endnutzer
Spezifisch	`details` pro Feld bei Validierungsfehlern
Kein Stacktrace	Interne Details nie an den Client

Speaker notes

Ein konsistentes Fehlerformat macht APIs leichter nutzbar. Maschinenlesbare Felder wie `type` oder `code` helfen bei automatischer Verarbeitung; menschenlesbare Nachrichten helfen beim Debugging. Interne Details wie Stacktraces gehören nicht in externe Fehlermeldungen.

Representations and Caching

Leitfrage: Wie liefert HTTP dieselbe Ressource in verschiedenen Formaten aus, und wie vermeidet man dabei unnötiges Übertragen?

Speaker notes

Eine Ressource und ihre Repräsentation sind unterschiedliche Konzepte: die Ressource ist die abstrakte fachliche Entität, die Repräsentation ist die konkrete Auslieferung in HTML, JSON, XML oder einem anderen Format. Content Negotiation lässt Client und Server aushandeln, welche Variante übertragen wird. Caching auf mehreren Ebenen (Browser, CDN, Anwendungscache) und Conditional Requests mit ETag/If-None-Match machen wiederholte Auslieferung effizient. Redirects und asynchrone Workflows mit `202 Accepted` runden das Bild ab: HTTP ist ein Protokoll, das Flusssteuerung über Statuscodes und Header erlaubt — nicht über proprietäre Konventionen.

Medienrepräsentationen und MIME Types

Eine Ressource ist nicht gleich ihr Datenformat. Dieselbe Ressource kann als verschiedene **Medientypen** übertragen werden:

Medientyp	Bedeutung	Beispiel
`text/html`	HTML-Dokument	Produktseite im Browser
`application/json`	JSON-Daten	API-Response
`text/css`	Stylesheet	Gestaltung einer Seite
`application/javascript`	JavaScript-Code	Skriptdatei
`image/png`	PNG-Bild	Produktfoto
`application/pdf`	PDF-Dokument	Rechnung zum Download

Media Type = Typ/Subtyp

Beispiele: `text/html`, `application/json`, `image/svg+xml`

Der Medientyp beschreibt nicht die Ressource selbst, sondern die Form ihrer aktuellen Repräsentation.

Speaker notes

MIME Type bedeutet ursprünglich Multipurpose Internet Mail Extensions; im Web spricht man meist gleichbedeutend von Media Types. Der Media Type sagt Client und Infrastruktur, wie der Body interpretiert werden soll. Ohne korrekten Content-Type weiß ein Browser oder API-Client nicht verlässlich, wie die empfangenen Bytes zu lesen sind. Ein Browser behandelt `text/html` anders als `image/png` oder `application/json`; genau deshalb ist der Medientyp Teil der HTTP-Semantik und nicht nur Dekoration.

Verschiedene Clients, verschiedene Repräsentationen

Nicht jeder Client braucht dieselbe Darstellung derselben Ressource:

Client-Modell	Typische Repräsentation	Wofür geeignet?
Server-gerenderter Browser-Client	`text/html`	Der Server liefert direkt eine fertige HTML-Seite
JavaScript-Frontend / SPA	`application/json`	Das Frontend rendert die Oberfläche selbst
Mobile App	`application/json`	Die App verarbeitet Daten und baut die UI nativ
Download-Client	`application/pdf`, `image/png`	Datei- oder Medienausgabe

Kernidee:
Die fachliche Ressource bleibt gleich, aber die **Repräsentation** wird an den Bedarf des Clients angepasst.

Speaker notes

Die drei Client-Modelle nutzen dieselbe Ressource, aber unterschiedliche Repräsentationen: Ein server-gerenderter Browser-Client erwartet `text/html`, weil der Browser direkt anzeigt, was der Server liefert. Ein JavaScript-SPA oder eine mobile App erwartet `application/json`, weil sie die Darstellung selbst übernehmen — die Daten kommen als maschinenlesbare Struktur, die Rendering-Logik liegt im Client. Download-Clients (PDF-Export, Bilddienste) erwarten binäre MIME-Types. Der `Accept`-Header des HTTP-Requests teilt dem Server mit, welche Repräsentation der Client verarbeiten kann — Content Negotiation erlaubt dem Server, die passende Variante auszuliefern.

HTML vs. JSON als Repräsentation

HTML-Repräsentation

- Enthält Struktur und Inhalt für die direkte Anzeige im Browser
- Typisch für **serverseitig gerenderte** Anwendungen
- Media Type: `text/html`

JSON-Repräsentation

- Enthält strukturierte Daten statt fertiger Seiten
- Typisch für **APIs**, SPAs und mobile Clients
- Media Type: `application/json`

Wichtig:
HTML und JSON konkurrieren nicht zwingend. Beide können sinnvolle Repräsentationen **derselben Ressource** sein.

Speaker notes

Serverseitig gerendertes HTML bedeutet, dass der Server die sichtbare Seite zusammensetzt. JSON ist dagegen vor allem eine Datenrepräsentation, die von Browser-JavaScript, mobilen Apps oder anderen Diensten weiterverarbeitet wird.

JSON kurz eingeführt

JSON steht für **JavaScript Object Notation**. Es ist ein leichtgewichtiges Textformat für strukturierte Daten.

```json
{
  "id": 42,
  "name": "Funkkopfhörer",
  "price": 79.99,
  "in_stock": true,
  "tags": ["audio", "wireless"],
  "manufacturer": {
    "name": "Acme",
    "country": "DE"
  },
  "discount": null
}
```

JSON-Element	Beispiel
Objekt	`{ "id": 42 }`
Array	`[1, 2, 3]`
String	`"name"`
Zahl	`79.99`
Boolean	`true`, `false`
Null	`null`

Speaker notes

JSON ist im Web so verbreitet, weil es kompakt, textbasiert und in fast allen Programmiersprachen leicht verarbeitbar ist. Für REST ist aber wichtig: JSON ist nur eine mögliche Repräsentation, nicht die Definition von REST.

Content Negotiation: Grundidee

Header	Richtung	Bedeutung
`Content-Type`	Request oder Response	Welches Format der Body hat
`Accept`	Request	Welche Formate der Client akzeptiert
`Accept-Language`	Request	Bevorzugte Sprache
`Accept-Encoding`	Request	Bevorzugte Kompression, z. B. gzip oder br
`Content-Encoding`	Response	Welche Kompression auf den Body angewendet wurde

```
GET /products/42 HTTP/1.1
Accept: application/json
Accept-Language: de
Accept-Encoding: gzip, br
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Language: de
Content-Encoding: gzip
```

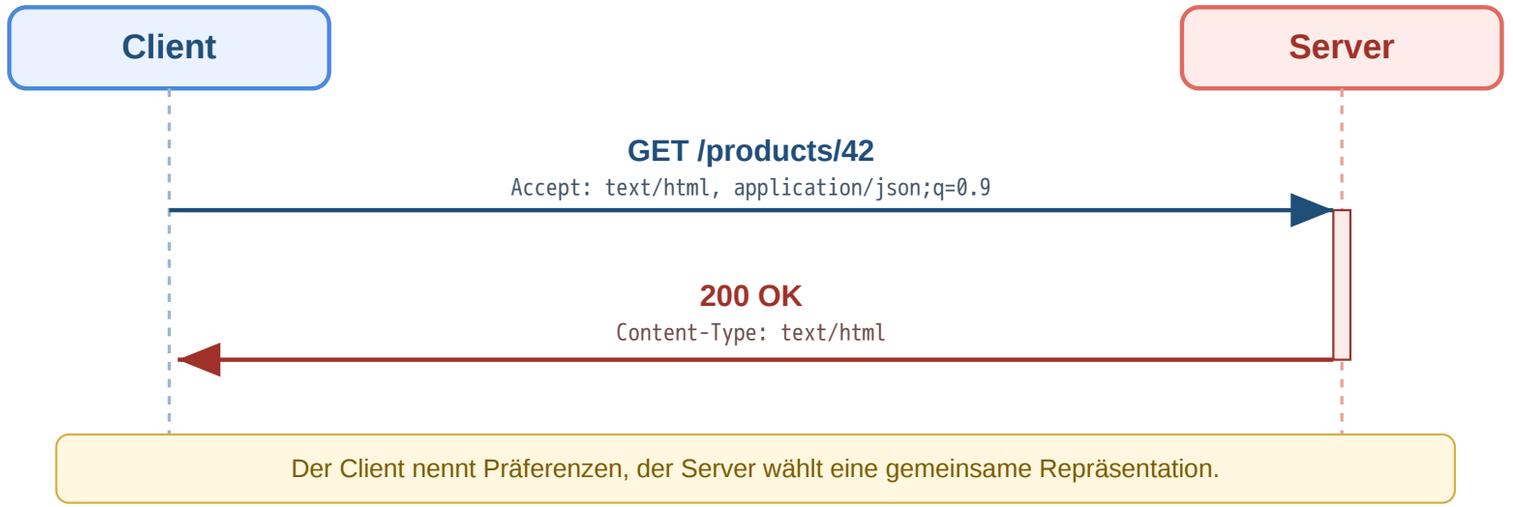


Speaker notes

Content Negotiation bedeutet, dass der Client Präferenzen mitteilt und der Server eine passende Repräsentation wählt. `Accept` beschreibt das gewünschte Format, `Content-Type` das tatsächlich gelieferte Format. Dasselbe Muster gibt es auch für Sprache und Kompression.

Content Negotiation: Repräsentation auswählen

Ein Client kann mehrere Formate akzeptieren und Prioritäten angeben:



Teil	Bedeutung
`text/html`	Bevorzugte Repräsentation
`application/json;q=0.9`	Ebenfalls akzeptabel, aber etwas weniger bevorzugt
`application/xml;q=0.5`	Nur niedrige Priorität

q-Wert = Qualitätswert
Je höher der q-Wert, desto stärker die Präferenz des Clients.
Wenn kein gemeinsames Format gefunden wird, kann der Server z. B. mit `406 Not Acceptable` antworten.

Speaker notes

Diese Form der Aushandlung erklärt, wie ein Client aktiv Einfluss auf die Darstellung nimmt. Für die Lehrlogik ist wichtig: Nicht der Pfad allein bestimmt die Repräsentation, sondern oft das Zusammenspiel aus Ressource, Headern und Serverentscheidung.

Eine Ressource, mehrere Repräsentationen

Beispiel: Dieselbe Produkt-Ressource `/products/42`

Browser-orientiert

```
GET /products/42 HTTP/1.1
Accept: text/html
```

```
HTTP/1.1 200 OK
Content-Type: text/html
```

API-orientiert

```
GET /products/42 HTTP/1.1
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Kernidee:
Die **Ressource** bleibt dieselbe, aber ihre **Repräsentation** kann je nach Client unterschiedlich sein.



Speaker notes

Die Trennung von Ressource und Repräsentation ist das Kernkonzept hinter dem R in REST — „Representational State Transfer". Eine Ressource (z.B. Produkt mit ID 42) ist eine abstrakte fachliche Entität; ihre Repräsentation ist das konkrete Format, in dem ihr Zustand übertragen wird (JSON, HTML, XML, CSV). Dieselbe Ressource kann mehrere Repräsentationen haben — der `Content-Type`-Header identifiziert, welche ausgeliefert wurde. Caches müssen diese Unterscheidung kennen, weil sie nicht nur URL, sondern auch Repräsentation für das Caching berücksichtigen (HTTP `Vary`-Header).

Warum das für HTTP-Verständnis wichtig ist

- `Content-Type` und `Accept` machen Repräsentationen explizit
- Caches müssen wissen, **welche Variante** einer Ressource gespeichert wurde
- Infrastruktur kann nur korrekt arbeiten, wenn Header die Nachricht eindeutig beschreiben
- Dieselbe fachliche Ressource kann für unterschiedliche Clients unterschiedlich dargestellt werden

> Header sind nicht Beiwerk, sondern Teil der HTTP-Semantik. Sie beschreiben Formate, Aushandlung, Kompression, Caching, Sicherheit und Zustand. Ohne dieses Verständnis bleiben spätere Themen wie Content Negotiation, Cache-Control oder API-Design unvollständig.

Speaker notes

Dieses Modul schafft die Brücke zwischen Grundsyntax und späteren Spezialfällen. Caching, Sessions, Sicherheit und später auch API-Design bauen darauf auf, dass Nachrichten und Repräsentationen durch Header praezise beschrieben werden. Besonders wichtig ist dabei die Unterscheidung zwischen fachlicher Ressource und uebertragener Variante: Ein Cache muss nicht nur wissen, dass `/products/42` gespeichert wurde, sondern auch ob dies die HTML-, JSON- oder komprimierte Variante war.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Speaker notes

Caching speichert Responses zwischen, um Last und Latenz zu reduzieren. `Cache-Control: max-age=300` erlaubt dem Browser, die Ressource für 300 Sekunden ohne Rückfrage wiederzuverwenden — der Request erreicht den Server in diesem Zeitfenster gar nicht. Dadurch kann der Nutzer kurzzeitig veraltete Daten sehen. Das ist kein Browser-Fehler, sondern exakt das beabsichtigte Verhalten auf Basis der vom Server gesetzten Direktive. Der Trade-off ist architektonisch explizit: Zu hohe `max-age`-Werte liefern veraltete Preise oder Lagerbestände; zu niedrige Werte erhöhen die Serverlast unnötig. `ETag`/`Last-Modified` mit Conditional Requests (`If-None-Match`, `If-Modified-Since`) bieten eine feinere Alternative: Der Cache wird validiert, ohne die vollständige Ressource erneut zu übertragen.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

> Der Browser hat die vorherige Response gecacht. Der `Cache-Control: max-age=300` Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

> Der Browser hat die vorherige Response gecacht. Der `Cache-Control: max-age=300` Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

> **Architekturentscheidung:** Wie lange sollen Produktdaten gecacht werden? Zu lang -> veraltete Preise. Zu kurz -> jeder Seitenaufruf belastet den Server.

Caching: Ebenen und Mechanismen

Cache-Ebene	Wo?	Vorteil
Browser-Cache	Lokal auf dem Gerät	Kein Netzwerk-Request -> schnellste Variante
Proxy/CDN-Cache	Edge-Server im Netzwerk	Reduziert Last auf Origin, niedrige Latenz
Application-Cache	Im Server (Redis, Memcached)	Vermeidet wiederholte Datenbankabfragen

Jede Ebene spart Arbeit an einer anderen Stelle:

- Browser-Cache spart Netzverkehr
- CDN spart Origin-Last
- Application-Cache spart Datenbank- oder Rechenaufwand

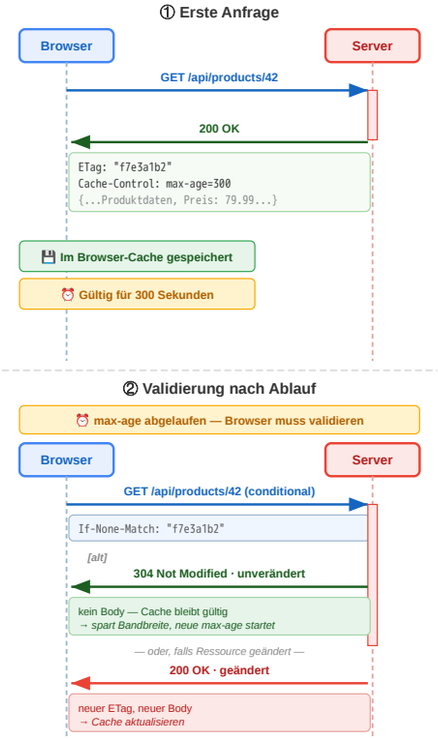
Speaker notes

Caches existieren auf mehreren Ebenen. Browser-Caches vermeiden Netzverkehr, CDN- oder Proxy-Caches entlasten den Ursprungsserver, serverseitige Application-Caches vermeiden teure Berechnungen oder Datenbankzugriffe. Alle drei Ebenen verbessern Performance, aber mit unterschiedlichen Trade-offs.

Cache-Steuerung über Header

Header	Funktion	Beispiel
`Cache-Control: max-age=3600`	1 Stunde gültig	Statische Assets
`Cache-Control: no-cache`	Immer validieren	Produktdaten, Preise
`Cache-Control: no-store`	Nie cachen	Bankdaten
`Cache-Control: public`	CDN darf cachen	Öffentliche Inhalte
`Cache-Control: private`	Nur Browser	Nutzerdaten
`ETag`	Ressourcen-Fingerprint	`"a1b2c3d4"`
`Last-Modified`	Letzter Änderungszeitpunkt	`Mon, 23 Mar 2026`

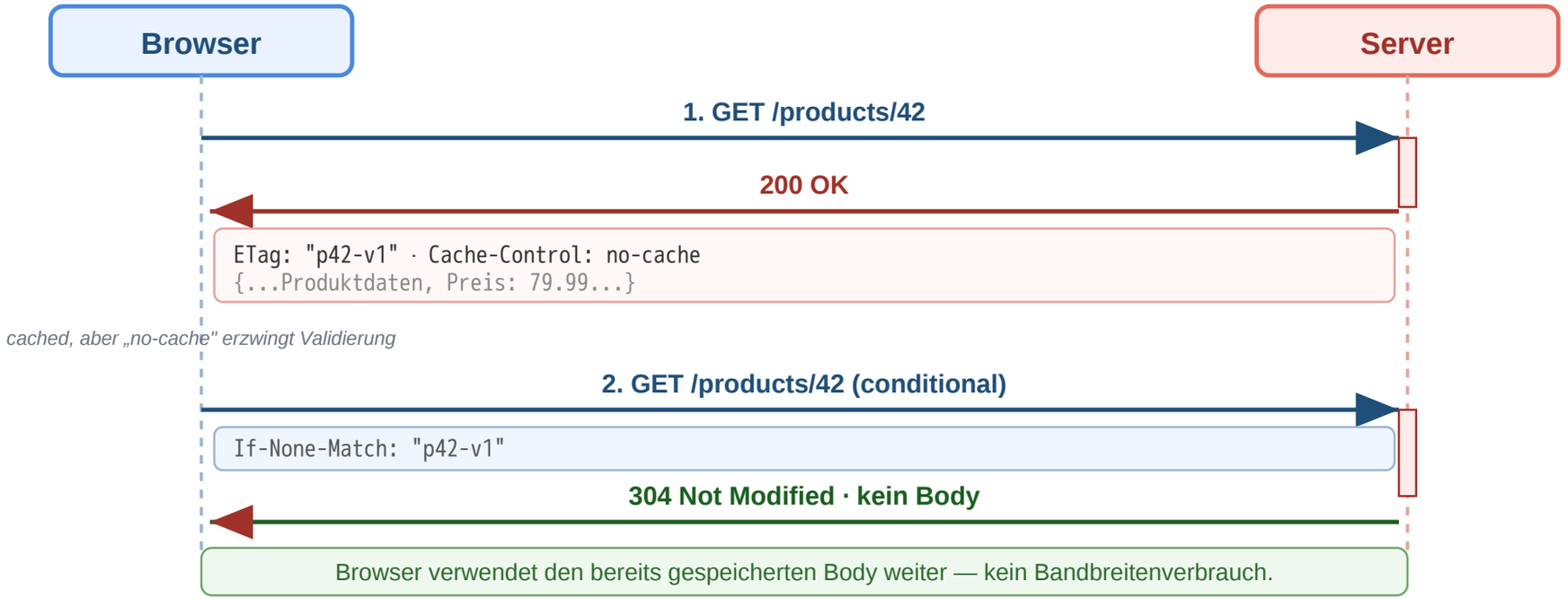
Wie der Browser entscheidet: Cache vorhanden? → max-age abgelaufen? → Validierungsrequest mit `If-None-Match: ETag`
Validierung: Server antwortet `304 Not Modified` (kein Body, spart Bandbreite) oder `200 OK` + neuer Body + neuer ETag.



Speaker notes

`Cache-Control` steuert, ob und wie lange eine Response gespeichert werden darf. `ETag` und `Last-Modified` ermöglichen Validierungsrequests. Bei einer erfolgreichen Validierung kann der Server `304 Not Modified` senden, sodass der Client seinen vorhandenen Body weiterverwendet.

Workflow: Conditional Request und Cache-Validierung



Schritt	Bedeutung
`ETag` empfangen	Client kennt Version der Repräsentation
`If-None-Match` senden	Client fragt: Hat sich etwas geändert?
`304 Not Modified`	Kein neuer Body nötig, Cache bleibt gültig

Was ist ein ETag? Ein **Entity Tag** ist ein vom Server vergebener, **opaker Versions-Fingerprint** einer konkreten Repräsentation. Format: in Anführungszeichen, z.B. `ETag: "a1b2c3d4"`. Der Client interpretiert ihn nie — er vergleicht nur.

Wie wird er berechnet? Die Wahl liegt beim Server; er muss nur garantieren, dass *gleicher Inhalt → gleicher ETag* und *geänderter Inhalt → anderer ETag*. Übliche Strategien:

Strategie	Beispiel	Wann sinnvoll
Inhalts-Hash	`MD5`/`SHA-1`/`xxHash` über den Response-Body	Statische Dateien, deterministisch generierte JSON-Responses
Metadaten-Kombination	`mtime + size + inode` (z.B. Apache, nginx)	Dateisystem-basierte Auslieferung — billig, kein Lesen nötig

Strategie	Beispiel	Wann sinnvoll
Datenbank-Version	`updated_at`-Timestamp oder `rev`-Spalte	API-Ressourcen aus DB — sehr effizient, kein Hashing

Stark vs. schwach: `"a1b2c3d4"` (strong) = byte-identisch; `W/"a1b2c3d4"` (weak, mit `W/`-Prefix) = semantisch äquivalent (z.B. Whitespace-Unterschiede irrelevant). Strong-ETags sind Pflicht für Range-Requests, weak reichen für Cache-Validierung.

Speaker notes

Dieser Workflow zeigt die typische Cache-Validierung für veränderliche Ressourcen. Der Client speichert eine Repräsentation mit `ETag` und fragt später bedingt erneut an. Wenn die Version unverändert ist, spart `304 Not Modified` Bandbreite, weil kein neuer Body gesendet werden muss. Der `ETag` selbst ist eine Server-interne Designentscheidung: Ein Hash über den Body ist semantisch am sauberstem, kostet aber CPU; eine Metadaten-Kombination wie `mtime+size` ist billig, kann aber bei identischem Inhalt unterschiedliche ETags erzeugen (Cache-Miss); eine DB-`updated_at`-Spalte oder ein monoton steigender Revisions-Counter ist für API-Ressourcen meist die beste Wahl, weil ohnehin verfügbar. Wichtig ist die Unterscheidung zwischen **strong** und **weak** ETags: Strong garantiert byte-für-byte-Gleichheit der Repräsentation und ist Voraussetzung für Range-Requests (partielles Nachladen eines Downloads); weak (`W/"…"`) erlaubt geringe Variationen wie unterschiedliche JSON-Whitespaces oder Komprimierung und reicht für die typische Cache-Validierung vollkommen aus. Der Client bleibt davon unberührt — er behandelt ETags als undurchsichtige Zeichenkette und vergleicht sie nur mittels `If-None-Match`. MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/ETag und https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/If-None-Match und https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/304

Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen `Cache-Control`-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
`app.v3.2.1.js` (versionierte JS-Datei)	
`/api/products/42` (Produktdaten mit Preis)	
`/api/cart` (Warenkorb des eingeloggten Nutzers)	
`logo.png` (Shop-Logo)	

Speaker notes

Die richtige Cache-Strategie hängt vom Datentyp ab. Versionierte statische Dateien sind nahezu ideal für lange Cache-Zeiten. Personalisierte oder sensible Daten sollten nicht im gemeinsamen Cache landen. Veränderliche Daten profitieren oft von Validierung statt unbedingter Frische.

Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen `Cache-Control`-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
`app.v3.2.1.js` (versionierte JS-Datei)	
`/api/products/42` (Produktdaten mit Preis)	
`/api/cart` (Warenkorb des eingeloggten Nutzers)	
`logo.png` (Shop-Logo)	

Ressource	Header	Begründung
`app.v3.2.1.js`	`public, max-age=31536000, immutable`	Version im Dateinamen → ändert sich nie
`/api/products/42`	`public, no-cache` + ETag	Preise können sich ändern, Validierung nötig
`/api/cart`	`private, no-store`	Nutzerspezifisch, darf nicht im CDN landen
`logo.png`	`public, max-age=86400`	Ändert sich selten, 1 Tag reicht

Redirects und das Post/Redirect/Get-(PRG)-Pattern

Ziel: Nach `POST /orders` zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: *"Formular erneut senden?"* — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch *Zurück → Vor* oder das Speichern als Lesezeichen löst dieses Verhalten aus.

Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.

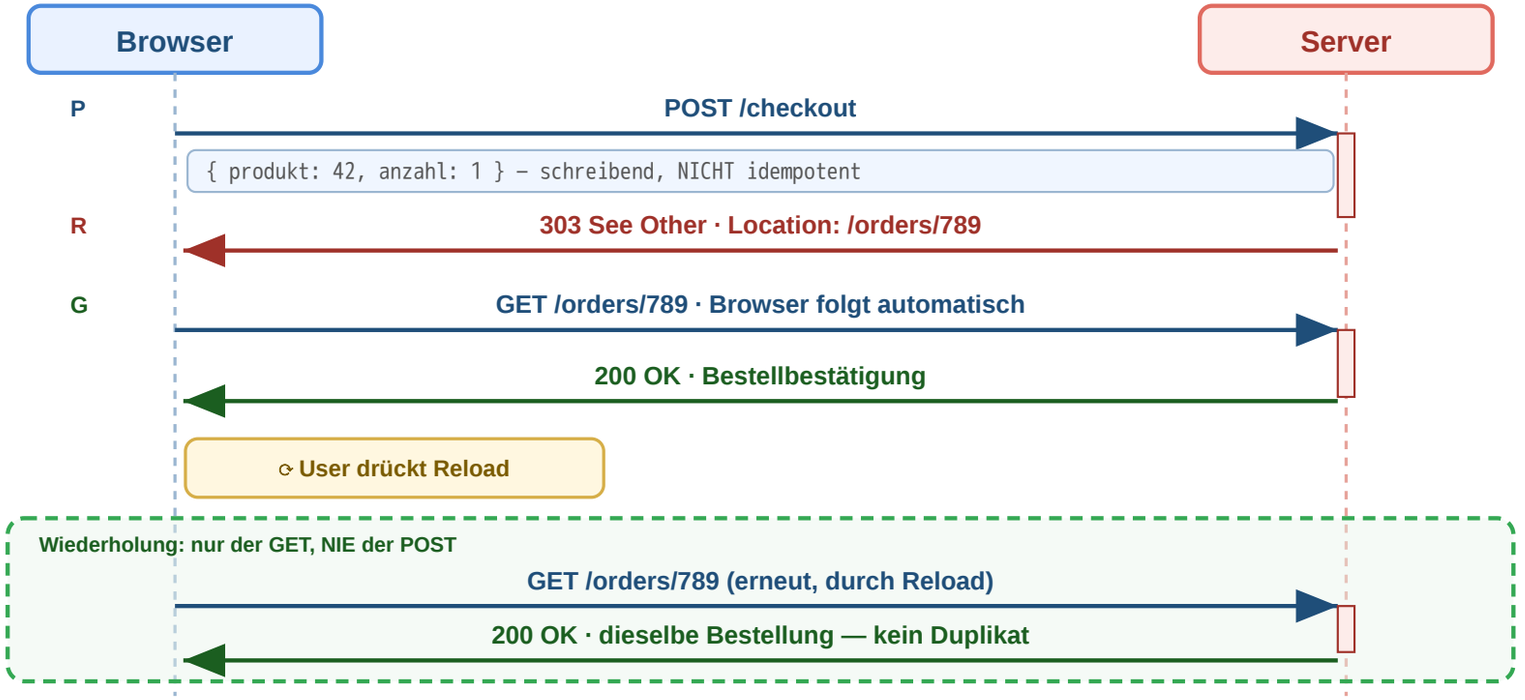
Speaker notes

Redirects verweisen Clients auf eine andere URI. `303 See Other` ist besonders wichtig nach einem schreibenden POST, weil der Browser anschließend per GET auf eine bestaetigende Ressource wechselt. Dieses Muster heißt Post/Redirect/Get und verhindert doppelte Formularverarbeitung beim Aktualisieren der Seite.

Redirects und das Post/Redirect/Get-(PRG)-Pattern

Ziel: Nach `POST /orders` zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: *"Formular erneut senden?"* — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch *Zurück → Vor* oder das Speichern als Lesezeichen löst dieses Verhalten aus.

Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.



Code	Semantik	Typischer Einsatz
301	Permanent verschoben	Domain-Umzug
302/307	Temporär woanders	A/B-Test, Wartung
303	Nach POST auf GET	**Post/Redirect/Get (PRG)**
308	Permanent, Methode bleibt	API-Migration

Post/Redirect/Get (PRG) verhindert doppelte Bestellungen: Nach POST antwortet der Server mit `303 See Other` → der Browser folgt per GET → Neuladen der Seite wiederholt nur den GET, nicht den POST.

Workflow: Redirects in der Praxis

	Methode darf wechseln *POST→GET in der Praxis historisch üblich*	**Methode & Body bleiben** *strikt nach RFC — wichtig für APIs*

Permanent
Caches und Crawler sollen URL ersetzen
	301 Moved Permanently — Form-URL dauerhaft umgezogen
	`→ POST /kontaktformular`
	`{ name, email, nachricht }`
	`← 301 Moved Permanently`
	`Location: /v2/kontakt`
	`→ GET /v2/kontakt ⚠ POST→GET!`
	`← 200 OK (Body verloren)`
	→ Browser wechselt POST→GET — Form-Daten weg!
	Bei reinem GET (z.B. HTTP→HTTPS) bleibt GET. Für POST-APIs: 308.
		308 Permanent Redirect — API-Migration v1 → v2
		`→ POST /api/v1/orders`
		`{ produkt: 42, anzahl: 1 }`
		`← 308 Permanent Redirect`
		`Location: /api/v2/orders`
		`→ POST /api/v2/orders { produkt: 42 }`
		`← 201 Created`
		→ Methode UND Body bleiben — POST bleibt POST.
		Clients sollten neue URL persistent übernehmen.

Temporär
URL nur jetzt anders, später wieder Original
	302 Found — POST bei abgelaufener Session
	`→ POST /api/order (Session weg)`
	`{ produkt: 42, anzahl: 1 }`
	`← 302 Found`
	`Location: /login`
	`→ GET /login ⚠ POST→GET!`
	`← 200 OK · Login-Formular`
	→ POST-Daten weg — Bestellung muss nach Login neu eingegeben werd
	Für temporäre Verlegung von Schreib-Endpoints daher 307.
		307 Temporary Redirect — Wartung · Failover
		`→ POST /api/orders`
		`{ produkt: 42 }`
		`← 307 Temporary Redirect`
		`Location: https://b.api.example/orders`
		`→ POST b.api.example/orders { produkt: 42 }`
		`← 201 Created`
		→ Methode & Body bleiben — Originaladresse merken.
		Ideal für Wartungsfenster, Failover, A/B auf POST.

Sonderfall 303 See Other — erzwingt nach POST einen GET zur Bestätigungsseite (Post/Redirect/Get).

„Dein POST war erfolgreich, hol die Antwort hier per GET" — macht Reload unschädlich (siehe vorige Folie).
Anders als 307/308 wechselt die Methode hier bewusst auf GET — auch wenn das Original ein POST war.

Ziel: Eine Ressource ist nicht (mehr) unter der angefragten URL erreichbar — etwa nach Domain-Umzug, URL-Bereinigung, A/B-Test, Sprach-/Geo-Weiterleitung oder als Antwort auf einen schreibenden POST. Der Client soll **automatisch zur richtigen URL geleitet** werden, ohne dass der Nutzer eingreifen muss; Suchmaschinen und Caches sollen die Verschiebung korrekt interpretieren.
Wir wollen: mit dem passenden Statuscode steuern, ob die Weiterleitung dauerhaft oder temporär ist und ob Methode und Body erhalten bleiben.

Speaker notes

Die Matrix macht zwei voneinander unabhängige Dimensionen sichtbar: **Permanenz** (cachebar, von Suchmaschinen übernommen) versus **Methodensemantik** (darf der Client bei der Weiterleitung von POST auf GET wechseln, oder muss Methode und Body erhalten bleiben). 301 ist der Klassiker für HTTP→HTTPS und Domain-Umzüge; Browser folgen mit GET und Crawler aktualisieren ihren Index. 302 ist die typische temporäre Weiterleitung, etwa zur Login-Wand oder für Sprach-/Geo-Routing — historisch uneinheitlich, weil viele Implementierungen POST→GET wechselten, obwohl die RFC das nicht vorschreibt. 307 und 308 wurden eingeführt, weil diese Inkonsistenz für APIs nicht akzeptabel ist: Beide garantieren, dass Methode und Body unverändert weitergereicht werden — 307 temporär, 308 permanent. 303 ist der Sonderfall, der das PRG-Muster der vorigen Folie erst möglich macht: Anders als 301/302 schreibt 303 explizit vor, dass der nächste Request ein GET sein muss — der Server signalisiert damit „dein POST war erfolgreich, hol die Antwort hier per GET". MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/303 und https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Redirections

Lang laufende Tasks: `202 Accepted` und Polling

Was tut der Server, wenn die Bearbeitung Sekunden oder Minuten dauert (CSV-Export, Video-Transcoding, LLM-Batch-Job)? Die Verbindung offen halten ist teuer und unzuverlässig. Das Standard-Muster ist ein **asynchroner Job mit Status-Polling** — eine Verkettung der bekannten Redirect-Bausteine.

```
# 1) Client startet den Task
POST /api/exports HTTP/1.1
Content-Type: application/json

{ "format": "csv", "range": "2026-Q1" }

# 2) Server quittiert SOFORT, ohne zu warten
HTTP/1.1 202 Accepted
Location: /api/jobs/abc123
Retry-After: 5

# 3) Client pollt den Job-Status (alle ~5 s)
GET /api/jobs/abc123 HTTP/1.1

HTTP/1.1 200 OK
Content-Type: application/json
{ "status": "processing", "progress": 0.42 }

# ⋯ weitere Polls ⋯

# 4) Fertig → Server schickt 303 zum Ergebnis
HTTP/1.1 303 See Other
Location: /api/exports/abc123.csv

# 5) Client folgt automatisch (PRG-Mechanik)
GET /api/exports/abc123.csv HTTP/1.1

HTTP/1.1 200 OK
```

Baustein	Funktion
`202 Accepted`	„Auftrag angenommen, Bearbeitung läuft"
`Location`	URL der **Job-Ressource** (nicht des Ergebnisses!)
`Retry-After`	Empfohlenes Poll-Intervall in Sekunden
Job: `200` + `status`	Zwischenstand: `queued`/`processing`/`done`/`failed`
Job: `303 See Other`	Endzustand → `Location` zeigt auf das fertige Artefakt

Wann nutzt man das?

- Server-seitig teure Berechnungen (Reports, ML-Inference, Renderings)
- Asynchrone Pipelines, die ohnehin in einem Worker laufen
- LLM-Batch-APIs (Anthropic, OpenAI) — gleiche Mechanik

Verwandte Muster

- *Long Polling*: Server hält die Anfrage offen, bis sich etwas ändert
- *WebSocket / SSE*: persistente Verbindung, Server pusht Events
- *Webhook*: Client gibt Callback-URL an, Server ruft zurück

Faustregel: Sobald die Bearbeitung länger als ein Browser-Timeout (≈ 30 s) dauern kann, in Job-Ressource auslagern und per Polling beobachten.

Speaker notes

Das Muster ist alt und bewährt — es trennt die Frage „Auftrag angekommen?" (sofortige Antwort) von „Auftrag fertig?" (kann lange dauern) sauber. Der entscheidende Punkt: Die `Location` aus dem `202 Accepted` zeigt *nicht* auf die fertige Ressource, sondern auf eine *Job-Ressource*, die ihrerseits einen eigenen Lebenszyklus hat (queued → processing → done/failed). Das ist konzeptionell wichtig, weil die Job-Ressource cachebar, abbruchbar (`DELETE /api/jobs/abc123`) und mit eigenen Statuscodes adressierbar ist. Der `303 See Other` am Ende ist dieselbe PRG-Mechanik wie bei einem normalen Form-Submit — der Client folgt mit GET zur fertigen Repräsentation. Vorsicht beim Polling-Intervall: Zu kurz erzeugt unnötige Last (jeder Poll ist ein voller Round-Trip mit Auth-Overhead), zu lang verzögert die wahrgenommene Antwortzeit. Server geben deshalb über `Retry-After` einen Hinweis. Echtes *Long Polling* ist eine Variante, bei der der Server die GET-Anfrage *offen hält*, bis sich der Job-Status ändert (oder ein Timeout greift); spart Round-Trips, blockiert aber Server-Verbindungen — heute meist durch SSE oder WebSockets ersetzt. **Beispiele aus der Praxis:** Anthropics Message Batches API, OpenAIs Batch API, GitHub Actions Job-Status, AWS S3 Multipart Uploads, Stripes asynchrone Webhooks — alle folgen einer Variante dieses Musters.

HTTP Sessions and CORS

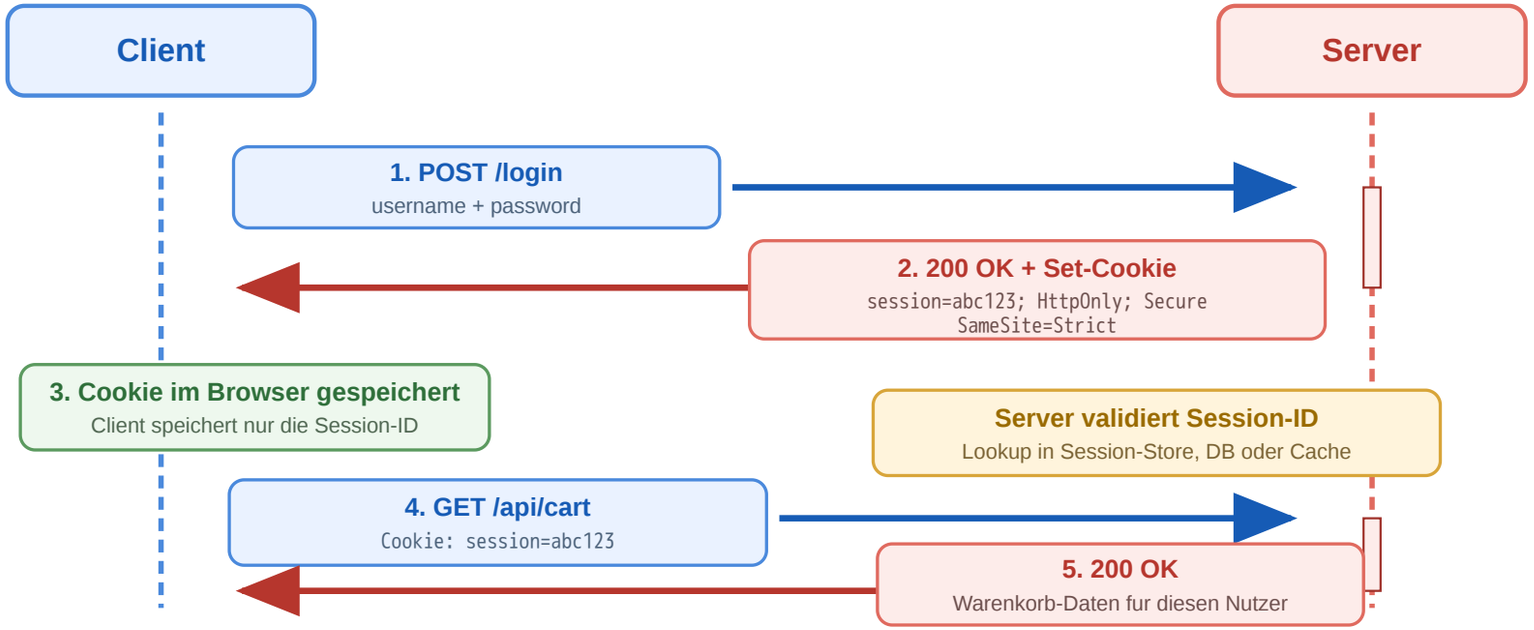
Leitfrage: Wie ermöglicht ein zustandsloses Protokoll persistente Sitzungen — und wie kontrolliert der Browser Cross-Origin-Zugriffe zwischen verschiedenen Domains?

Speaker notes

HTTP ist zustandslos, aber Cookies geben Clients eine wiedererkennbare Identität, an die Server Sessions hängen. Same-Origin isoliert Origins voneinander und ist die Sicherheitsbasis sowohl für Cookies (jede Domain liest nur ihre eigenen) als auch für JavaScript (kein Zugriff auf DOM und Daten anderer Origins). CORS lockert diese Isolation kontrolliert für berechtigte Cross-Origin-Anwendungsfälle — moderne SPAs gegen separate API-Domains funktionieren nur dank CORS. Beide Mechanismen sind grundlegend für die meisten heutigen Web-Architekturen.

Sessions und Cookies

HTTP ist zustandslos — aber der Online-Shop braucht Login und Warenkorb. **Cookies** lösen diesen Widerspruch:



Ablauf

1. `POST /login` mit Credentials → Server prüft.
2. Server antwortet mit `Set-Cookie: session=abc123` und legt die Session (Warenkorb, User-ID, …) serverseitig ab.
3. Browser speichert das Cookie für die Domain und sendet bei jedem weiteren Request automatisch `Cookie: session=abc123` mit.
4. Server liest die ID, schlägt die Session nach und kennt damit Identität und Zustand des Clients.

> Das Cookie enthält meist nur eine **opake ID** — der eigentliche Zustand liegt im Server (DB oder Redis).

Fundament: Same-Origin Policy — eine *Origin* ist die Kombination aus **Schema + Host + Port** (z.B. `https://shop.example:443`). Cookies gehören zu genau einer Origin:

- Der Browser sendet ein Cookie **nur zurück an die Domain, die es gesetzt hat** — nie an andere Domains.
- JavaScript auf `evil.example` kann das Cookie von `shop.example` **nicht lesen** (`document.cookie` ist origin-isoliert).
- Aber: bindet `news.example` ein Bild von `tracker.ad.net` ein, sendet der Browser dabei das Cookie von `tracker.ad.net` mit — ohne dass `news.example` es jemals sieht. Genau diese Regel ermöglicht Third-Party-Tracking (s. spätere Folien) und ist gleichzeitig die Basis, auf der die Angriffe der nächsten Folie (XSS, CSRF) ansetzen.

Speaker notes

HTTP selbst ist zustandslos. Cookies führen dennoch wiedererkennbare Clients ein, indem der Browser kleine Wertepaar-Informationen speichert und bei späteren Requests mitsendet. Bei serverseitigen Sessions enthält das Cookie meist nur eine Session-ID; der eigentliche Zustand liegt auf dem Server. Das hat zwei Konsequenzen: Erstens ist die ID das einzige, was ein Angreifer braucht, um sich als der Nutzer auszugeben — ihr Schutz ist sicherheitskritisch. Zweitens muss der Server den Session-Store irgendwo halten; in horizontal skalierten Setups führt das zu Sticky Sessions oder einem geteilten Backend wie Redis.

Die **Same-Origin Policy** ist eine der wichtigsten Sicherheitsregeln des Browsers überhaupt. Sie isoliert Origins voneinander: Skripte einer Origin dürfen weder DOM noch Cookies einer anderen Origin lesen. Cookies sind dabei *domain-gebunden* (nicht origin-gebunden im strengen Sinne — Cookies kennen Schema/Port nicht so feingranular wie der Rest der Policy, deshalb gibt es zusätzlich das `Secure`-Attribut). Wichtig ist die Asymmetrie zwischen *Lesen* und *Senden*: Lesen ist strikt origin-isoliert, aber das automatische *Mitsenden* von Cookies bei eingebetteten Ressourcen oder Cross-Site-Requests passiert auch jenseits der eigenen Origin — und genau das ist die Lücke, die CSRF und Third-Party-Tracking ausnutzen. Die nachfolgenden Schutzmechanismen `SameSite`, `HttpOnly` und `Secure` schließen diese Lücke an unterschiedlichen Stellen.

Angriffsvektoren auf Session-Cookies — und Schutz

Das Cookie ist die Identität. Wer die Session-ID besitzt, ist für den Server der Nutzer — kein Passwort nötig. Drei Wege, an die ID zu kommen, drei Schutzmechanismen am Cookie-Attribut.

Drei Angriffe

Angriff	Wie?	Schutz-Attribut
XSS (Cross-Site Scripting)	Eingeschleustes JS liest `document.cookie` und exfiltriert die Session-ID an einen fremden Server.	`HttpOnly` — Cookie ist für JS unsichtbar
CSRF (Cross-Site Request Forgery)	Bösartige Site löst im Hintergrund einen Request an die Bank/Shop-API aus; Browser sendet Cookie automatisch mit.	`SameSite=Strict` / `Lax` — Cookie wird bei Cross-Site-Requests nicht gesendet
MITM / Sniffing	Cookie wird im Klartext über HTTP übertragen; im offenen WLAN abgreifbar.	`Secure` — Cookie wird nur über HTTPS gesendet

Sichere Konfiguration

Konfiguration	XSS	CSRF	Sniffing
`session=abc123`	✗	✗	✗
`…; HttpOnly`	✓	✗	✗
`…; HttpOnly; Secure`	✓	✗	✓
`…; HttpOnly; Secure; SameSite=Strict`	✓	✓	✓

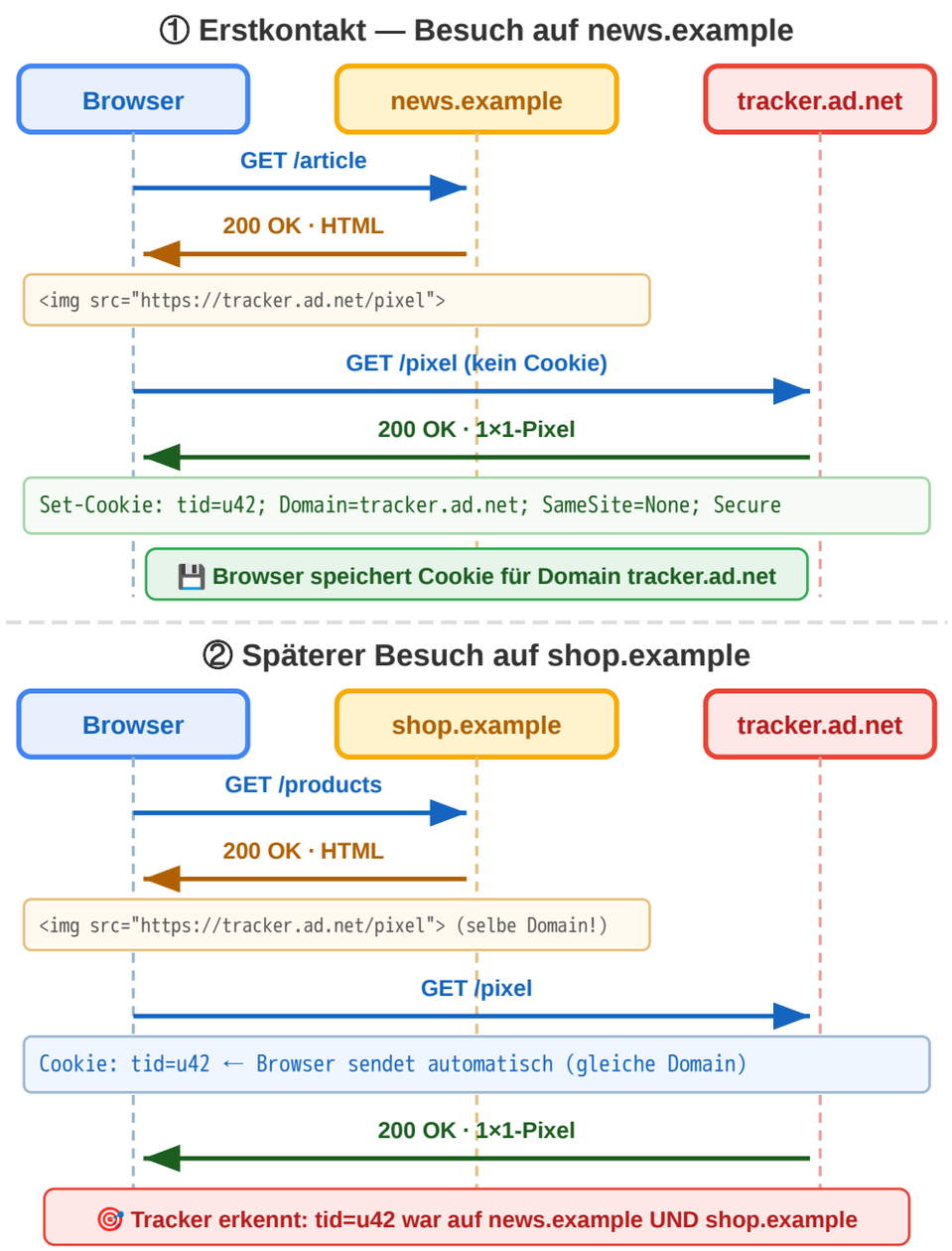
```
Set-Cookie: session=abc123;
  HttpOnly; Secure; SameSite=Strict;
  Path=/; Max-Age=3600
```

Zusätzlich: Session-ID nach Login **rotieren** (gegen Session-Fixation) · Session serverseitig **invalidieren** bei Logout · kurze **Max-Age** + Sliding-Renewal.

Speaker notes

Session-Cookies sind ein klassisches Beispiel, bei dem Sicherheit nicht im Anwendungscode entsteht, sondern in der korrekten Konfiguration eines Protokoll-Mechanismus. Die drei Angriffsvektoren adressieren drei unterschiedliche Bedrohungsmodelle: **XSS** unterstellt, dass ein Angreifer Code im Browser des Opfers ausführt — z.B. über einen unsicheren Forum-Beitrag oder ein unsicheres Eingabefeld; `HttpOnly` macht das Cookie für JavaScript unsichtbar und schließt den direkten Diebstahl aus. **CSRF** unterstellt, dass der Nutzer auf einer fremden Site landet, die einen Request an die geschützte Anwendung auslöst — der Browser sendet das Session-Cookie automatisch mit, weil es zur Zieldomain gehört; `SameSite=Lax` oder `Strict` verhindert genau das. **Sniffing** unterstellt einen passiven Angreifer im Netzwerk; `Secure` zwingt den Browser, das Cookie nur über TLS zu senden. Wichtig ist, dass diese drei Attribute orthogonal sind — jedes blockiert genau einen Vektor, und nur die Kombination aller drei plus serverseitige Hygiene (ID-Rotation nach Login, Logout-Invalidierung, kurze Laufzeiten) ergibt eine belastbare Session-Implementierung. Wer Single-Page-Apps mit Cross-Site-API-Calls baut, muss zusätzlich CSRF-Token oder ein Double-Submit-Pattern einsetzen, weil `SameSite=Strict` dort zu restriktiv wäre.

Cookie-Tracking I — Third-Party-Cookies



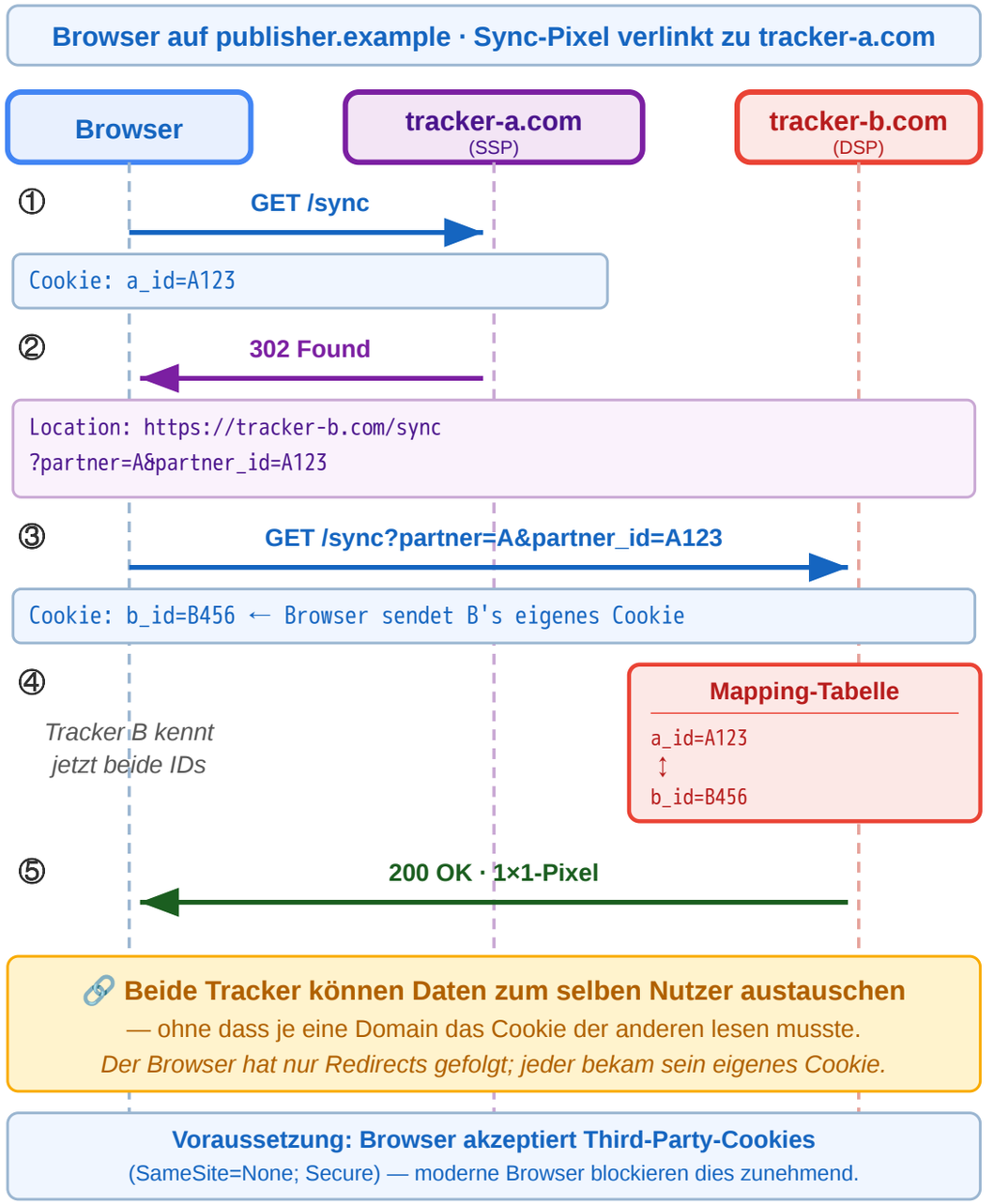
1. Mehrere Websites binden eine Ressource derselben Tracking-Domain ein (Bild, Skript, Werbe-Pixel, Social-Plugin).
2. Beim ersten Kontakt setzt der Tracker ein Cookie für **seine eigene Domain** — z.B. `tracker.ad.net`.
3. Bei jedem weiteren Besuch einer Site, die dieselbe Tracker-Ressource einbindet, sendet der Browser das Cookie automatisch mit.
4. Der Tracker erkennt: derselbe Browser war auf news.example **und** shop.example.

> Same-Origin bleibt gewahrt: Jede Domain liest nur ihre eigenen Cookies. Tracking funktioniert dadurch, dass viele Sites **dieselbe** Tracker-Domain einbinden — nicht weil Cookies geteilt werden.

Speaker notes

Third-Party-Cookies sind die einfachste Form von Cross-Site-Tracking. Der Trick ist nicht, dass Cookies geteilt werden — der Browser sendet Cookies nur an genau die Domain, die sie gesetzt hat. Der Trick ist, dass viele unabhängige Websites Ressourcen von derselben Tracker-Domain einbinden (Werbenetz, Analytics, Social-Buttons). Aus Sicht des Trackers entsteht so ein zusammenhängendes Profil über Sites hinweg, obwohl die Sites selbst nichts voneinander wissen. Moderne Browser (Safari ITP, Firefox ETP, Chrome Privacy Sandbox) beschränken Third-Party-Cookies inzwischen aggressiv — wer trotzdem trackt, weicht auf Mechanismen wie Cookie-Syncing oder First-Party-Bounce-Tracking aus.

Cookie-Tracking II — ID-Sync über Redirects



Akteure: Im Online-Werbemarkt vermitteln zwei Arten von Plattformen zwischen Publisher und Werbetreibendem:

- **SSP (Supply-Side Platform)** — auf der Verkäuferseite. Vermarktet die Werbeflächen eines Publishers (z.B. einer Nachrichtensite) und versteigert sie in Echtzeit an den höchstbietenden Käufer.
- **DSP (Demand-Side Platform)** — auf der Käuferseite. Kauft im Auftrag von Werbetreibenden gezielt Werbeflächen, um deren Zielgruppen anzusprechen.

Beide haben jeweils eigene Nutzer-IDs in eigenen Cookies — kennen aber die ID des Partners nicht. Same-Origin verbietet das direkte Lesen.

Lösung — Cookie-Syncing:

1. Sync-Pixel der SSP wird geladen; SSP liest ihr Cookie (`a_id=A123`).
2. SSP antwortet mit `302 Redirect` zur DSP — die eigene ID steckt **im URL-Parameter** (`?partner_id=A123`).
3. Browser folgt zur DSP und sendet dabei **deren eigenes Cookie** (`b_id=B456`) mit; in der URL liegt A's ID.
4. DSP speichert die Zuordnung `A123 ↔ B456` in ihrer Mapping-Tabelle und kann den Nutzer fortan beim Bidding wiedererkennen.

Speaker notes

Cookie-Syncing (auch *Cookie-Matching* oder *ID-Matching*) ist der Mechanismus hinter Real-Time-Bidding und Werbenetzen. Er löst genau das Problem, dass eine Site die Cookie-ID einer Tracker-Domain nicht kennt: Statt das Cookie direkt zu teilen, wird die ID als URL-Parameter über eine Redirect-Kette zur Partner-Domain weitergereicht. Der Browser folgt jedem Redirect automatisch und sendet dabei zur jeweiligen Zieldomain deren eigenes Cookie. So bekommt der Empfänger zwei Informationen gleichzeitig: die fremde ID aus dem URL-Parameter und seine eigene aus dem mitgeschickten Cookie — und kann beide in einer Mapping-Tabelle verknüpfen. Same-Origin wird formal eingehalten; die Identitätsverknüpfung ist aber faktisch erfolgt. Dieser Mechanismus erklärt auch, warum Privacy-Maßnahmen wie Third-Party-Cookie-Blockaden, SameSite-Defaults und Bounce-Tracking-Erkennung in den letzten Jahren verschärft wurden.

CORS — Same-Origin kontrolliert lockern

Warum es Same-Origin überhaupt gibt: Der Browser sendet Cookies *automatisch* an die Domain, die sie gesetzt hat. Ohne weitere Schutzregel könnte eine beliebige bösartige Site `evil.example` einfach `fetch("https://meinebank.example/konto")` aufrufen. Same-Origin Policy verhindert das: Skripte einer Origin dürfen Responses anderer Origins **nicht lesen**. Das ist die Sicherheitsbasis, auf der Cookies, Logins und sensible APIs im Browser überhaupt funktionieren können.

Problem: Genau dieselbe Regel blockiert auch legitime Anwendungsfälle. Eine SPA auf `https://app.example` will per `fetch()` mit `https://api.example` sprechen — beide gehören demselben Anbieter, sind aber unterschiedliche Origins. Ohne Ausnahme würde der Browser jeden Request blockieren.

CORS (*Cross-Origin Resource Sharing*) ist die kontrollierte Lockerung: Der **Server** entscheidet per Response-Header, welche fremden Origins lesend zugreifen dürfen. Der Browser erzwingt die Regel — das Skript bekommt das Response-Objekt nur dann zu sehen, wenn der Server zustimmt.

Zwei Arten von Cross-Origin-Requests:

Typ	Auslöser	Verhalten
Simple	`GET`/`HEAD`/`POST` mit nur Standard-Headern und Standard-Content-Types (`text/plain`, `multipart/form-data`, `application/x-www-form-urlencoded`)	Browser schickt den Request direkt; bei fehlendem `Access-Control-Allow-Origin` *verwirft* er die Antwort vor dem Skript.
Preflight-pflichtig	`PUT`/`DELETE`/`PATCH`, `Content-Type: application/json`, eigene Header wie `Authorization`	Browser schickt erst einen `OPTIONS`-Preflight (siehe nächste Folie) und nur bei

Server-Antwort (vereinfacht):

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://app.example
Content-Type: application/json

{ "user": "anna", ... }
```

Ohne `Access-Control-Allow-Origin` blockiert der Browser den Zugriff aus JavaScript (sichtbar als CORS-Fehler in der Konsole) — der Server kann dabei trotzdem schon einen Treffer protokolliert haben, weil der Request selbst ankam. CORS schützt also den *Client* vor unbeabsichtigtem Datenabfluss an fremde Origins, nicht den Server vor Anfragen.

Wichtig: CORS gilt nur für **Browser-JS** (`fetch`/`XHR`). Server-zu-Server-Requests, `curl`, oder Native-Apps sind nicht betroffen — die haben keine Same-Origin Policy.

Speaker notes

Die Same-Origin Policy ist die fundamentale Browser-Sicherheitsregel: Skripte einer Origin dürfen weder DOM noch Daten anderer Origins lesen. CORS ist die Ausnahmeregel, mit der Server explizit Zugriff erlauben — ohne sie würden moderne SPA-Architekturen (Frontend auf Vercel, API auf eigener Domain) nicht funktionieren. Die Asymmetrie ist wichtig: CORS schützt den Client (genauer: den User, dessen Browser eine Drittsite besucht) davor, dass JavaScript dieser Drittsite unkontrolliert mit fremden Origins reden und deren Daten an die Drittsite zurückgeben kann. Den *Server* schützt CORS nicht — wer einen Endpunkt vor unautorisierten Zugriffen schützen will, braucht Authentifizierung. Die Unterscheidung simple vs. preflight ist historisch: simple Requests entsprechen dem, was schon HTML-Formulare per `POST` konnten, also vor CORS-Zeiten ohnehin möglich war. Alles, was darüber hinausgeht (eigene Methoden, JSON-Bodies, Auth-Header), erfordert vorherige Erlaubnis per Preflight.

CORS: Preflight mit `OPTIONS`

Ziel: Bevor der Browser einen Request schickt, der *mehr kann als ein klassisches HTML-Formular* (`PUT`/`DELETE`, JSON-Body, eigene Header wie `Authorization`), fragt er den Server erst per `OPTIONS`, ob die Operation überhaupt erlaubt ist. **Warum:** Simple Requests waren schon vor CORS möglich (HTML-Formular kann jede URL anfragen) — die kann der Browser nicht mehr zurückhalten. Alles darüber hinaus ist *neue* Fähigkeit, die XHR/`fetch` erst geschaffen hat — der Server muss sie deshalb explizit freigeben, bevor der Browser sie an eine fremde Origin schickt. So vermeidet man, dass eine bösartige Site neue Cross-Origin-Operationen am Server auslöst.

① Browser fragt nach:

```
OPTIONS /v1/orders HTTP/1.1
Host: api.example
Origin: https://app.example
Access-Control-Request-Method: PUT
Access-Control-Request-Headers:
  Authorization, Content-Type
```

② Server antwortet:

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Methods: PUT, DELETE
Access-Control-Allow-Headers:
  Authorization, Content-Type
Access-Control-Max-Age: 86400
```

③ Nur bei passender Antwort schickt der Browser anschließend den eigentlichen `PUT /v1/orders` los — sonst wird er ohne Netzwerkversuch direkt im JS abgebrochen.

Was der Browser konkret prüft:

Antwort-Header	Prüfung
`Allow-Origin`	matched die aufrufende Origin?
`Allow-Methods`	enthält die geplante Methode (`PUT`)?
`Allow-Headers`	enthält alle nicht-Standard-Header (`Authorization`, eigene)?

`Max-Age: 86400` — der Browser cacht die Preflight-Antwort 24 h und spart sich `OPTIONS` für Folgeaufrufe. Policy-Änderungen greifen erst nach Cache-Ablauf.

Speaker notes

Der Preflight-Mechanismus ist der Hauptgrund für CORS-Probleme in der Praxis: Entwickler vergessen oft, dass jeder JSON-`POST` oder authentifizierte Request eine vorausgehende `OPTIONS`-Anfrage auslöst, die der Server auch beantworten muss. Wer einen API-Endpunkt schreibt, muss neben dem eigentlichen Handler auch einen `OPTIONS`-Handler bereitstellen oder ein Framework verwenden, das CORS automatisch handhabt (Express `cors`, Spring `@CrossOrigin`, FastAPI `CORSMiddleware`, …). Bei der Auswahl der `Allow-Origin`-Werte gilt: lieber explizite Liste der erlaubten Origins als ein pauschales `*`, weil `*` keine Cookies erlaubt (siehe nächste Folie) und außerdem ein zu weites Vertrauensfenster aufmacht. In großen Setups mit vielen Frontend-Domains wird `Origin` request-spezifisch geprüft und der Header dann mit genau dem zurückgegeben, was angefragt wurde — `Vary: Origin` muss dabei gesetzt sein, damit Caches die Antworten nicht über Origins hinweg vermischen.

CORS und Credentials

> **Default:** Cross-Origin-`fetch` sendet **keine Cookies** und keine `Authorization`-Header — auch wenn der Browser sie für die Zieldomain gespeichert hätte. Das schützt vor unbeabsichtigtem Mitsenden von Anmeldedaten an fremde Origins.

Damit Cookies/Auth mitgehen — drei Bedingungen müssen alle erfüllt sein:

1. Client signalisiert Absicht:

```
fetch("https://api.example/me", {
  credentials: "include"   // statt default "same-origin"
})
```

2. Server erlaubt explizit:

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
```

> **Wildcard verboten:** `Access-Control-Allow-Origin: *` ist mit `Allow-Credentials: true` **nicht** kombinierbar. Die Origin muss explizit genannt werden, sonst lehnt der Browser ab.

3. Cookie selbst muss cross-site-fähig sein:

Das Zusammenspiel mit `SameSite`:

`SameSite`	**Verhalten bei Cross-Origin**
`Strict`	Cookie wird **nie** an fremde Origin gesendet — auch bei expliziter CORS-Freigabe nicht
`Lax` (default)	Cookie geht nur bei *Top-Level-Navigation* mit, nicht bei `fetch`/XHR
`None` + `Secure`	Cookie wird bei Cross-Origin gesendet, sofern CORS zustimmt

> **Typisches Setup für SPA + getrennte API-Domain:**
>
> - Frontend `app.example`, API `api.example`
> - Session-Cookie: `SameSite=None; Secure; HttpOnly`
> - CORS auf API: `Allow-Origin: https://app.example` + `Allow-Credentials: true`
> - Frontend-`fetch` mit `credentials: "include"`
>
> Alternativ — wenn man `SameSite=Strict` halten will: Frontend und API auf derselben Site (z.B. `app.example` + `app.example/api/*`) hosten oder ein BFF-Pattern verwenden.

```
Set-Cookie: session=abc123;
  HttpOnly; Secure;
  SameSite=None     ← Pflicht für Cross-Origin
```

Warum so streng? Ohne diese Einschränkungen könnte eine bösartige Seite per `fetch` Login-Cookies des Nutzers an authentifizierte APIs mitschicken — das alte CSRF-Problem in CORS-Form. Die Dreifach-Zustimmung (Client + Server + Cookie) macht das praktisch unmöglich.

Speaker notes

Cookies und CORS-Credentials sind der am häufigsten missverstandene Bereich von CORS. Die Regel `Allow-Origin: *` kombiniert mit `Allow-Credentials: true` wäre eine ernsthafte Sicherheitslücke — jede beliebige Origin könnte authentifizierte Requests im Namen des Nutzers stellen — und der Browser lehnt diese Kombination deshalb hart ab. Wer `Allow-Credentials: true` braucht, muss die erlaubte Origin per Request bestimmen und mit `Vary: Origin` korrekt cachen lassen. Das Zusammenspiel mit `SameSite` ist die zweite Falle: viele Entwickler setzen `SameSite=None; Secure`, um Cross-Origin-Zugriffe zu ermöglichen, und merken erst spät, dass damit auch der gesamte CSRF-Schutz auf der Cookie-Ebene wegfällt. Wer mit Cookies arbeitet, sollte daher in modernen SPA-Setups das BFF-Pattern bevorzugen: das Frontend spricht mit einem eigenen Backend auf derselben Site, das wiederum nach hinten mit der eigentlichen API-Domain kommuniziert. Damit bleiben Cookies First-Party und SameSite kann strikt bleiben. Die OAuth-/JWT-Welt umgeht das Cookie-Problem teilweise durch Bearer Token im `Authorization`-Header — die unterliegen ebenfalls CORS, aber ohne das `SameSite`-Detail.

HTTP Authentifizierung und Sicherheit

Leitfrage: Wie schützt HTTP Identität, Vertraulichkeit und Integrität auf der Leitung — und welche Mechanismen autorisieren API-Zugriff in unterschiedlichen Szenarien?

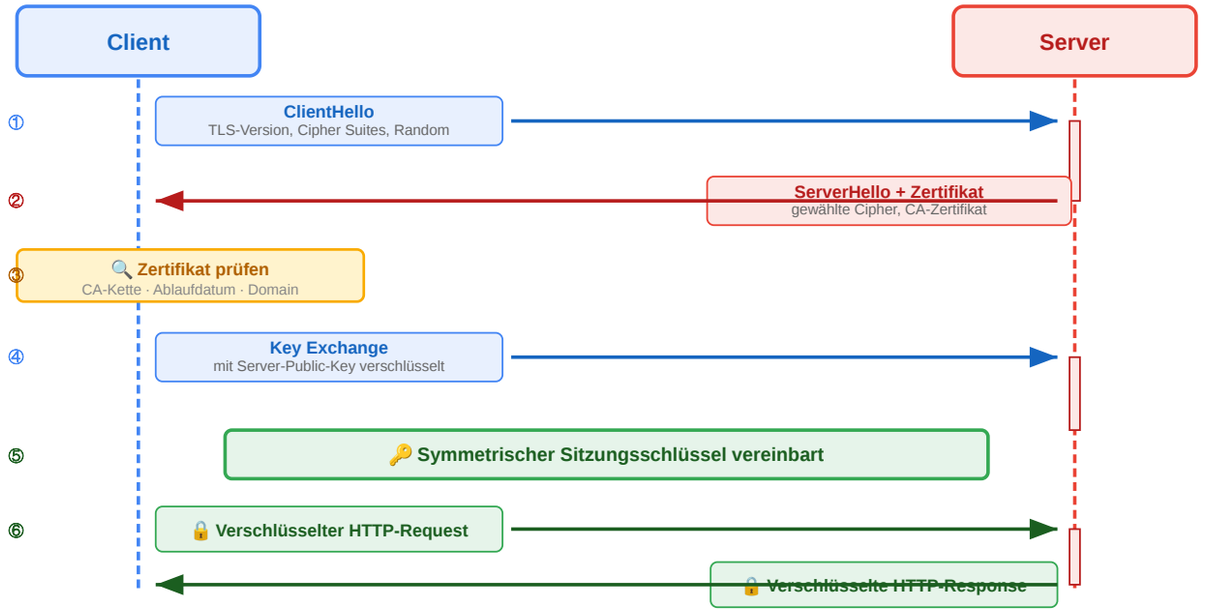
Speaker notes

TLS sichert den Transport gegen Mitlesen und Manipulation; Authentifizierungsmechanismen identifizieren den Client gegenüber dem Server. Beides zusammen ergibt die Sicherheitsschicht über reinem HTTP. Vier dominante Authentifizierungs-Mechanismen — API Key, Session Cookie, OAuth 2.0, JWT — adressieren unterschiedliche Bedrohungsmodelle und Skalierungsanforderungen. Welcher passt, hängt vom Szenario ab: Maschine-zu-Maschine, Browser-Session, delegierte Drittapp-Zugriffe, stateless Microservices.

HTTPS und TLS

Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt

Diskussionsfrage: Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Wie schützt TLS hier und was schützt es nicht?



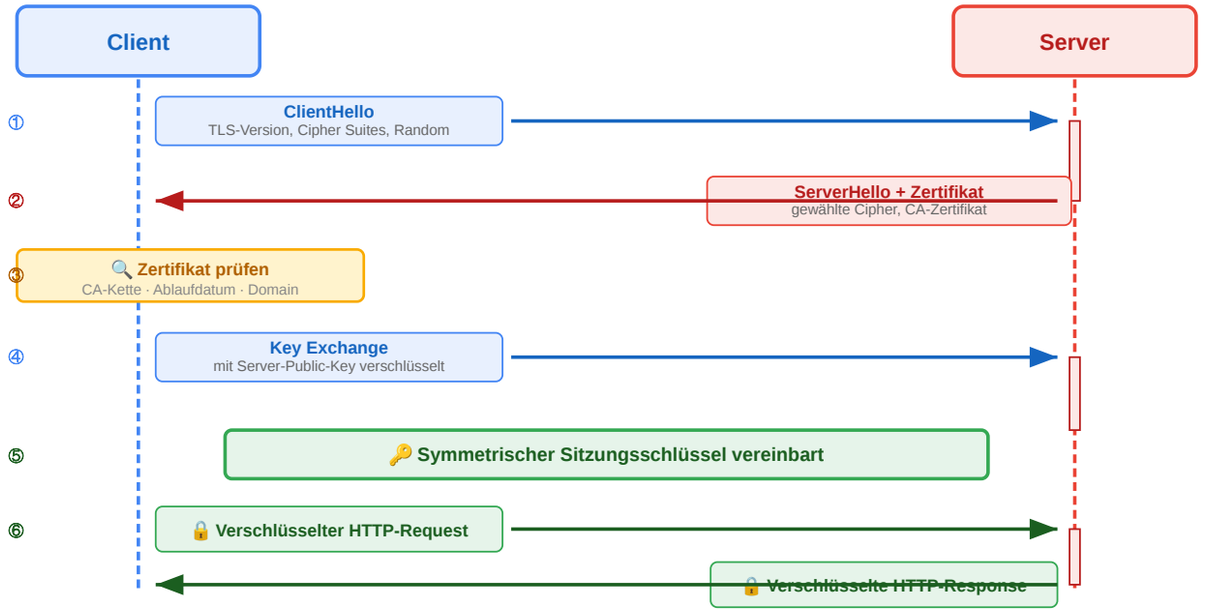
Speaker notes

HTTPS bedeutet HTTP über TLS (Transport Layer Security). TLS schützt Vertraulichkeit durch Verschluesselung, Integrität durch Manipulationsschutz und Authentizität durch Zertifikate. Es schützt die Transportstrecke, aber nicht automatisch den Browser selbst oder unsichere Anwendungslogik. Die Diskussionsfrage zielt darauf, dass Studierende den Unterschied zwischen passiven (Sniffing) und aktiven (MITM) Angriffen verstehen — und erkennen, dass Authentizität die *Voraussetzung* für Vertraulichkeit ist, nicht etwas Eigenständiges. Wer dem TLS-Endpunkt nicht vertraut, hat auch die Verschlüsselung verloren.
Wie TLS die drei Eigenschaften herstellt: Der TLS-Handshake (links visualisiert) verläuft in drei Phasen. Erst tauschen Client und Server ihre Cipher-Suites und ephemeren Schlüsselanteile aus (ECDHE — Elliptic-Curve Diffie-Hellman) und einigen sich auf einen gemeinsamen *Session Key*, ohne dass dieser je über die Leitung geht. Parallel präsentiert der Server sein **X.509-Zertifikat**, das eine Zertifizierungsstelle (CA) signiert hat; der Client prüft die Signaturkette gegen sein im Betriebssystem hinterlegtes Root-Store — das erzeugt **Authentizität**. Ab Phase drei verschlüsselt jede Seite den Anwendungstraffic mit dem Session Key symmetrisch (AES-GCM/ChaCha20-Poly1305) — das erzeugt **Vertraulichkeit**. Das gleiche AEAD-Verfahren hängt an jedes Paket einen Authentication Tag, der bei Manipulation auffällt — das erzeugt **Integrität**. Wichtig: TLS schützt *die Leitung*, nicht den Endpunkt. Ein kompromittierter Browser, eine XSS-Lücke oder ein unsicherer Cookie sind durch TLS nicht abgedeckt. Ebenso wenig Metadaten wie DNS-Lookup oder SNI — diese verraten weiter, *welche* Domain besucht wird (DoH/DoT und Encrypted Client Hello adressieren das, sind aber nicht universell verbreitet).

HTTPS und TLS

Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt

Diskussionsfrage: Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Wie schützt TLS hier und was schützt es nicht?



Lösung:

- **Vertraulichkeit** schützt — das WLAN-Sniffing sieht nur verschlüsselten Datenverkehr.
- **Integrität** schützt — manipulierte Pakete würden auffallen.
- **Authentizität** schützt im engeren Sinne nicht vor *passivem Mitlesen*, ist aber Voraussetzung dafür, dass der Schutz überhaupt greift: ohne Zertifikatsprüfung könnte ein aktiver Angreifer einen eigenen TLS-Endpunkt vorspielen (MITM) und die Verbindung im Klartext mitlesen.

Was TLS *nicht* schützt: DNS-Anfragen und Traffic-Muster sind weiter sichtbar — der Angreifer sieht *welche* Site besucht wird, nur nicht *was* übertragen wird.

Authentifizierung vs. Autorisierung — Überblick

> **Authentifizierung (AuthN):** *Wer bist du?* — Identitätsnachweis.
> **Autorisierung (AuthZ):** *Was darfst du?* — Berechtigung für eine konkrete Operation oder Ressource.

Mechanismus	Funktionsweise	State	Typischer Einsatz
API Key	Statischer Schlüssel im Header	Server-Lookup	M2M-APIs, CLI-Tools, Webhooks
Session Cookie	Opake Session-ID im Cookie + Server-Store	stateful	Klassische Web-Apps
Bearer Token (JWT)	Signierter Token mit Claims	stateless	SPAs, Mobile, Microservices
OAuth 2.0	Delegierter Token vom Authorization Server	je nach Flow	„Login mit Google", Third-Party-Integrationen

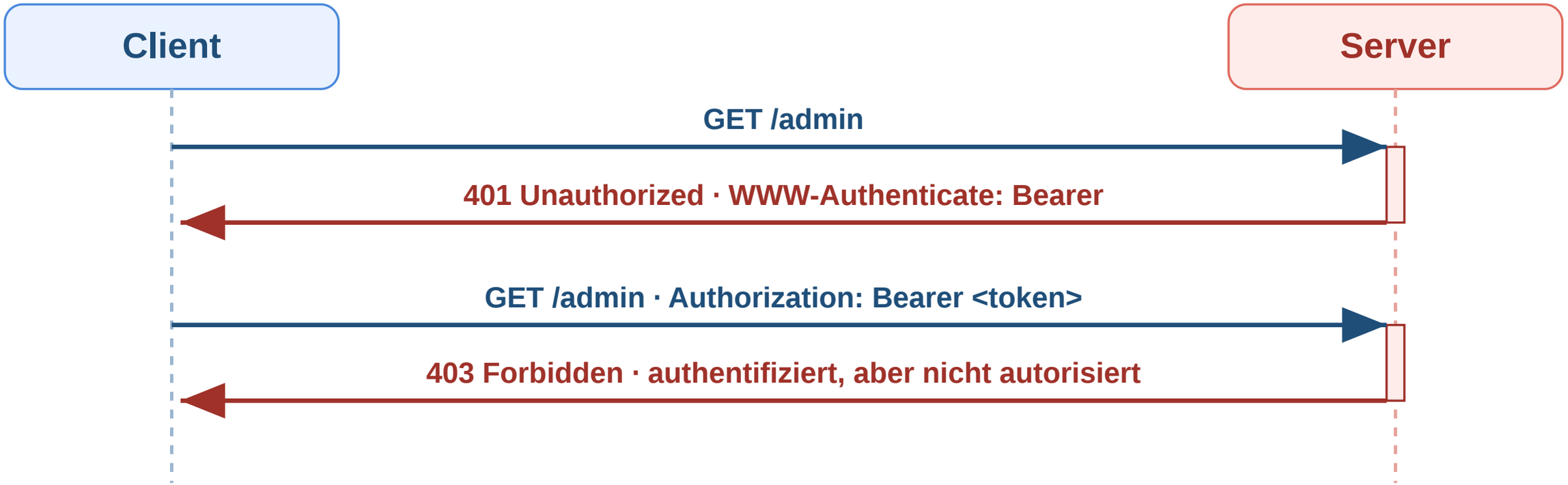
Die vier Mechanismen sind keine Alternativen für dasselbe Problem, sondern adressieren unterschiedliche Szenarien. Auf den folgenden Folien jeweils einer im Detail.

Speaker notes

Authentifizierung beantwortet die Frage, *wer* ein Client ist; Autorisierung beantwortet, *was* dieser Client tun darf. Beide Konzepte werden im Alltag häufig verwechselt — der Statuscode `401` adressiert AuthN, `403` adressiert AuthZ. Die hier gezeigten vier Mechanismen unterscheiden sich nicht nur technisch, sondern auch im Bedrohungsmodell: API Keys sichern Maschine-zu-Maschine-Verbindungen ohne Nutzerkonzept; Session Cookies stabilisieren browserbasierte Sitzungen mit serverseitigem Widerruf; JWTs übertragen identitätstragende Behauptungen ohne Server-State; OAuth delegiert Zugriff auf fremde Ressourcen, ohne dass Passwörter geteilt werden. Die folgenden Folien gehen auf jeden Mechanismus einzeln ein.



Workflow: Authentifizierung mit 401 und 403



Status	Bedeutung
`401 Unauthorized`	Keine gültigen Anmeldedaten vorhanden
`403 Forbidden`	Identität bekannt, aber Zugriff nicht erlaubt

Speaker notes

Der Unterschied zwischen `401` und `403` ist didaktisch zentral. `401 Unauthorized` fordert zunächst gültige Authentifizierung an und geht oft mit `WWW-Authenticate` einher. `403 Forbidden` bedeutet dagegen, dass die Identität akzeptiert wurde, aber für diese Operation keine Berechtigung besteht. MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/401 und https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/403

Mechanismus 1 — API Key

```
GET /v1/charges HTTP/1.1
Host: api.stripe.com
Authorization: Bearer sk_live_4eC39HqLyjWDarjtT1zdp7dc
```

Alternativ über eigenen Header:

```
GET /v1/charges HTTP/1.1
X-API-Key: sk_live_4eC39HqLyjWDarjtT1zdp7dc
```

Funktionsweise:

1. Provider stellt den Schlüssel einmal aus; Konsument speichert ihn sicher (Env-Variable, Secret-Manager).
2. Bei jedem Request schickt der Client den Schlüssel im Header mit.
3. Server schlägt den Schlüssel in einer DB nach, ermittelt das zugehörige Account und prüft Quotas und Scopes.

✓ Vorteil	✗ Nachteil
Trivial zu implementieren	Schlüssel = Vollzugriff
Funktioniert ohne Browser	Kein automatischer Ablauf
Cachebar, loggbar	Selten an Nutzer gebunden

Praxis: Stripe, GitHub PATs, OpenAI-API, Slack-Bots — überall dort, wo *Server* mit *Server* spricht und kein interaktiver Nutzer dahinter steht.

Schlüssel nie ins Repo, nie in URLs, nie in Logs. Bei Leak: sofort rotieren.

Speaker notes

Der API Key ist die einfachste Form von Authentifizierung — und genau deshalb für Maschine-zu-Maschine-Szenarien beliebt. Die Sicherheit hängt vollständig daran, dass der Schlüssel geheim bleibt: er sollte nie im Quellcode, in Versionskontrolle oder in URLs auftauchen, sondern in Umgebungsvariablen oder Secret-Stores (HashiCorp Vault, AWS Secrets Manager, …) liegen. Da es keinen automatischen Ablauf gibt, ist regelmäßige Rotation entscheidend; Provider wie Stripe bieten dafür Multi-Key-Mechanismen, damit ein neuer Schlüssel parallel zum alten existieren kann, bis alle Clients umgestellt sind.

Mechanismus 2 — Session Cookie

```
# 1) Login
POST /login HTTP/1.1
{ "username": "anna", "password": "..." }

# 2) Server-Response
HTTP/1.1 200 OK
Set-Cookie: session=abc123;
  HttpOnly; Secure; SameSite=Strict; Path=/

# 3) Folge-Request – Browser sendet automatisch
GET /dashboard HTTP/1.1
Cookie: session=abc123
```

Funktionsweise:

1. Login: Server prüft Credentials, legt Session-Eintrag in DB/Redis ab, schickt Session-ID im Cookie.
2. Browser speichert Cookie domain-gebunden und sendet es bei jedem weiteren Request mit.
3. Server schlägt ID im Session-Store nach → kennt Identität und Zustand.

✓ Vorteil	✗ Nachteil
Browser handhabt Cookie automatisch	Server muss Session-Store skalieren
Logout = sofortige serverseitige Invalidierung	Stateful — Sticky Sessions oder geteilter Cache
Mit `HttpOnly`/`Secure`/`SameSite` robust	Schlecht für Cross-Domain-APIs

> **Praxis:** Server-gerenderte Apps (Rails, Django, Laravel, Spring MVC) und moderne **BFF**-Architekturen ("Backend for Frontend"), bei denen das SPA-Frontend über einen eigenen Backend-Proxy gegen die API spricht.

Speaker notes

Session Cookies sind die klassische Form der Browser-basierten Authentifizierung. Ihr entscheidender operativer Vorteil gegenüber JWTs ist die volle Server-Kontrolle: Logout, Zwangsabmeldung, Berechtigungswechsel wirken sofort beim nächsten Request, weil dieser den aktuellen Session-Eintrag liest. Der Preis ist Statefulness — bei horizontaler Skalierung müssen alle App-Server denselben Session-Store sehen, üblicherweise via Redis oder Memcached. Die Sicherheits-Attribute (`HttpOnly`, `Secure`, `SameSite`) wurden auf der Cookie-Sicherheits-Folie behandelt; sie sind hier nicht optional, sondern zwingend.

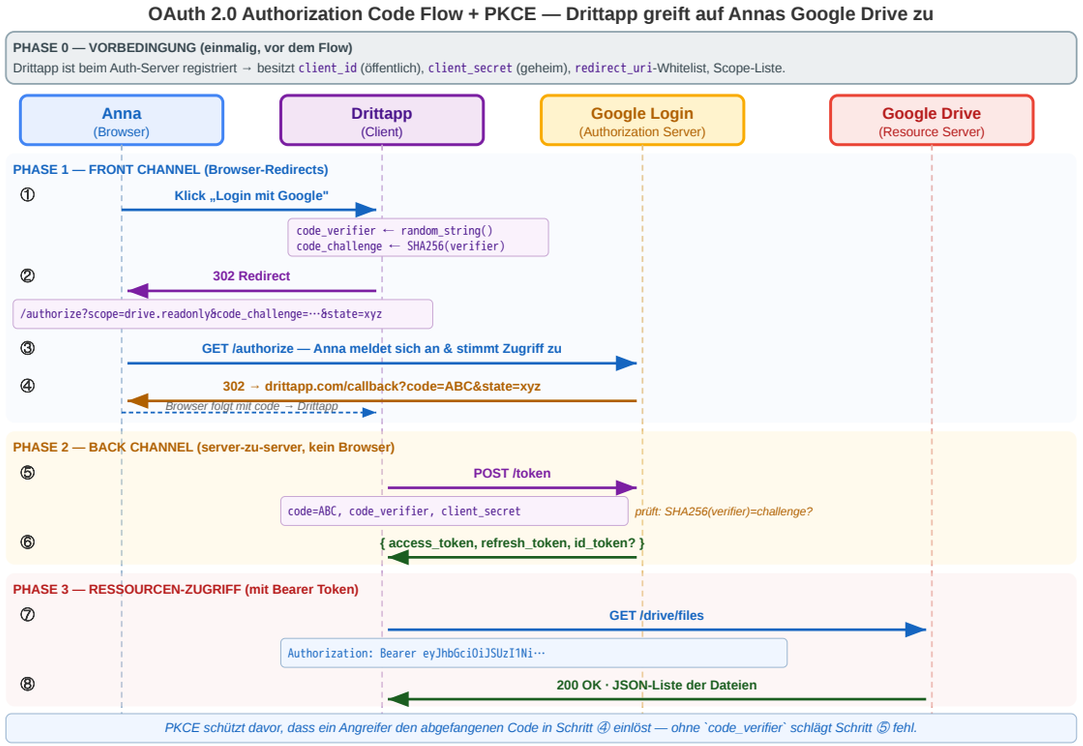
Mechanismus 3 — OAuth 2.0 (Überblick)

Problem: Eine App will im Auftrag des Nutzers auf eine *fremde* Ressource zugreifen (Kalender, Mails, Drive) — der Nutzer soll *nicht* sein Passwort weitergeben.

Vier Akteure:

Akteur	Rolle	Beispiel
Resource Owner	Der Nutzer	Anna
Client	App, die zugreifen will	Drittapp
Authorization Server	Vergibt Tokens	Google Login
Resource Server	Hält die Daten	Google Drive API

> **Wichtig:** OAuth 2.0 ist *Autorisierung* (Delegation), nicht primär *Authentifizierung*. Für „Login mit Google" wird **OIDC** verwendet, das auf OAuth aufsetzt und einen JWT als ID-Token zurückgibt. Details sind eigenes Thema.



Speaker notes

Konzept im Großen. OAuth 2.0 löst ein anderes Problem als die ersten beiden Mechanismen: nicht „Wie authentifiziert sich Client X bei Server Y?", sondern „Wie kann App A im Auftrag des Nutzers auf eine Ressource bei Server B zugreifen, ohne dass A die Credentials sieht?". Das Konzept der **delegierten Autorisierung** über einen separaten Authorization Server ist heute die Grundlage jeder Federated-Login-Erfahrung (Login mit Google/GitHub/Microsoft) und jeder API-Integration zwischen großen Plattformen. Der **Authorization Code Flow mit PKCE** ist der aktuelle Standard-Flow; ältere Flows wie *Implicit* oder *Resource Owner Password Credentials* gelten als deprecated. **OpenID Connect (OIDC)** ergänzt OAuth um eine echte Identity-Aussage (`id_token` als JWT) — was technisch erklärt, warum „Login mit Google" überhaupt funktioniert: OAuth allein liefert nur einen Access Token, *wer* der Nutzer ist, sagt es nicht.

Was ist PKCE — und was sind `code_verifier` und `code_challenge`? PKCE (*Proof Key for Code Exchange*, gesprochen „pixy", RFC 7636) ist ein Schutzmechanismus gegen Code-Interception. Die Drittapp erzeugt zu Beginn jedes Login-Flows ein zufälliges Geheimnis — den `code_verifier` (ein langer Zufallsstring, 43–128 Zeichen Base64-URL). Davon berechnet sie einen Hash: `code_challenge = SHA256(code_verifier)`, Base64-URL-codiert. Die Challenge wird in Phase 1 *öffentlich* an den Auth-Server geschickt (sie reist im URL-Parameter über den Browser), der Verifier bleibt im Drittapp-Server *geheim*. In Phase 2 schickt die Drittapp den Verifier dann *direkt* (Back-Channel) an den Auth-Server, der `SHA256(verifier) == challenge` prüft. Ein Angreifer, der den Code aus Phase 1 abfängt, hat nur die Challenge gesehen — den dazugehörigen Verifier kann er nicht rekonstruieren (One-way-Hash), kann den Code also nicht einlösen.

Was passiert in der Grafik (Schritt für Schritt):

- **Phase 0 — Client-Registrierung (oben als Banner; einmalig vor dem Flow):** Lange vor Annas Klick hat der Entwickler die Drittapp in der **Google Cloud Console** registriert. Google vergibt eine `client_id` (öffentlich, identifiziert die App), ein `client_secret` (nur dem Drittapp-Server bekannt), eine **Whitelist erlaubter** `redirect_uri`**-Werte** und die anfragbaren Scopes. Diese Registrierung ist der Grund, warum Google die Drittapp in Phase 1 überhaupt wiedererkennt — die `client_id` in jeder Anfrage referenziert genau diesen Eintrag, und die `redirect_uri` darf nur auf eine der hinterlegten URLs zeigen. Damit kann ein Angreifer keinen abgefangenen Code an eine eigene Domain umleiten lassen.

- **Phase 1 — Front-Channel (① – ④, über Annas Browser):** Anna klickt „Login mit Google". Die Drittapp erzeugt das PKCE-Paar (Verifier + Challenge) und schickt Anna mit einem `302 Redirect` zu Google — die URL enthält `client_id`, `scope=drive.readonly`, `code_challenge` und ein `state` (CSRF-Schutz). Annas Browser landet bei Googles `/authorize`-Endpunkt, sie meldet sich an und sieht den **Consent-Dialog** („Drittapp will Zugriff auf `drive.readonly` — Erlauben?"). Nach Zustimmung antwortet Google mit `302` zurück zu `drittapp.com/callback?code=ABC&state=⋯` — der **Authorization Code** ist kurzlebig (~30 s), one-time, und reist *über den Browser*.

- **Phase 2 — Back-Channel (⑤ – ⑥, server-zu-server, ohne Browser):** Die Drittapp ruft Googles `/token`-Endpunkt **direkt** auf — mit `code`, `code_verifier` (jetzt im Klartext) und `client_secret`. Google prüft: stimmt der `state`, ist die `redirect_uri` in der Whitelist, ist `SHA256(verifier) == challenge`? Wenn ja, kommt `{ access_token, refresh_token, id_token? }` zurück.

- **Phase 3 — Ressourcen-Zugriff (⑦ – ⑧):** Die Drittapp ruft die Drive-API mit `Authorization: Bearer <access_token>` und erhält die Dateiliste.

Warum der Umweg über zwei Kanäle? Der Browser sieht nur den `code` und die Challenge — nie den Verifier, nie das Access Token, nie das Client Secret. Selbst wenn ein Angreifer den Redirect-Parameter aus Phase 1 abfängt, kann er den Code ohne Verifier und Secret nicht einlösen. Das eigentliche Token entsteht in einer Server-zu-Server-Kommunikation, die nie über das Gerät des Nutzers läuft. Auf den folgenden zwei Folien klären wir, was so ein Access Token überhaupt ist — am Beispiel von JWTs.

Mechanismus 4 — Bearer Token (JWT)

Warum brauchen wir JWTs? OAuth (vorige Folie) liefert dem Client einen *Access Token* — aber was steckt eigentlich drin? In verteilten Architekturen (Microservices, Edge-Deployments, Multi-Region) muss jeder Service einen Aufrufer authentifizieren können, **ohne mit einer zentralen Session-DB zu sprechen**. JWTs lösen genau das: ein vom Auth-Server signiertes, selbstbeschreibendes Token, das alle Identitätsdaten selbst trägt — *Server-zu-Server-Authentifizierung* wird zur reinen lokalen Signaturprüfung, ohne Roundtrip zum Auth-Server.

JWT = *JSON Web Token* (RFC 7519). Drei Base64-URL-codierte Teile, durch Punkte getrennt — `Header.Payload.Signature`.

Teil	Inhalt
Header	Algorithmus (`alg`) und Token-Typ (`typ`)
Payload	*Claims*: `sub` (User-ID), `exp` (Ablauf), `iat`, `iss`, Rollen, Tenant — beliebige JSON-Felder
Signature	Kryptografische Signatur über `Header.Payload` mit Server-Secret (HMAC) oder Private Key (RSA/ECDSA)

Warum drei Teile, warum diese Codierung?

- **Base64-URL** statt Plain-JSON: macht den Token URL-/Header-sicher.
- **Signatur trennbar**: Server prüft nur den hinteren Teil; *was* signiert wurde, bleibt für Debugging lesbar.
- **Header trägt** `alg`: Der Verifier weiß *bevor* er den Inhalt parst, mit welchem Verfahren er prüfen muss.

Wie funktioniert die Verifikation — und warum reicht das?

1. Server berechnet aus `Header.Payload` + bekanntem Secret/Public-Key erneut die Signatur.
2. Vergleicht mit der mitgesendeten Signatur (timing-safe). Stimmt sie nicht → 401.
3. Liest Claims aus der Payload. Prüft `exp` (nicht abgelaufen?), `iss`/`aud` (vom richtigen Aussteller, für mich bestimmt?).
4. **Kein DB-Lookup nötig** — alle Identitätsdaten stecken im Token selbst.

Bei asymmetrischer Signatur (RSA/ECDSA) holt der prüfende Service den Public Key des Auth-Servers einmalig über **JWKS** (JSON Web Key Set, meist `/.well-known/jwks.json`) und cacht ihn. Er muss kein Geheimnis hüten.

Speaker notes

Diese Folie führt das Konzept ein. JWTs sind populär geworden, weil sie ein hartnäckiges Skalierungsproblem lösen: bei klassischen Session-Cookies muss jeder API-Server Zugriff auf denselben Session-Store haben, was in verteilten Architekturen operativ teuer wird. Ein JWT ist dagegen *self-contained* — die Identitätsbehauptungen reisen im Token selbst, jeder Service prüft die Signatur lokal und kennt sofort den Nutzer, ohne mit Redis oder einer User-DB sprechen zu müssen. Besonders mächtig wird das mit asymmetrischer Signatur: der Authorization Server hat den Private Key und ist die einzige Stelle, die Tokens erzeugen kann; alle Resource Server bekommen über JWKS den Public Key und können prüfen, *ohne dass sie irgendein Geheimnis hüten*. Genau dieses Trust-Modell macht JWTs zur Standardwährung in OAuth/OIDC-Flows. Die Detail-Trade-offs (Widerruf, Sicherheitsfallen) auf der nächsten Folie.

JWT — Beispiel, Nutzen und Trade-offs

Beispiel-Request:

```
GET /api/me HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.
  eyJzdWIiOiI0MiIsImV4cCI6MTcxNTAwMDAwMCwicm9sZSI6ImFkbWluIn0.
  Mz5e_kJ4qP···SIG
```

Decodiert (Base64-URL):

```
{"alg":"RS256","typ":"JWT"}                    // Header
{"sub":"42","exp":1715000000,"role":"admin"}   // Payload
```

Warum nützlich: horizontale Skalierung ohne Session-Store; weiterreichbar zwischen Microservices; asymmetrische Signatur → Public-Key-Verifikation überall, keine geteilten Geheimnisse.

Preis — Widerruf vor Ablauf ist schwer. Drei Muster:

Muster	Kerngedanke
Kurze `exp` + Refresh-Token	Access-Token 5–15 min; serverseitig invalidierbares Refresh-Token verlängert
Server-Blocklist	`jti` widerrufener Tokens in Redis prüfen — sofortige Wirkung, dafür wieder stateful
Key-Rotation	Signing Key tauschen — invalidiert *alle* Tokens, Notbremse

> **Faustregel:** Wer sofortigen, selektiven Widerruf braucht (Compliance, Finanzaufsicht, Healthcare), nimmt eher Session Cookies. JWTs sind für „eventually consistent" Identitätsaussagen gemacht, nicht für Echtzeit-Zugriffskontrolle.

Speaker notes

Beispiel und Nutzen. Das `Authorization: Bearer …`-Header-Beispiel zeigt einen typischen RS256-signierten Token (Header gibt `alg: RS256` an, Payload trägt Standard-Claims `sub`/`exp` plus Custom `role`). Der Server liest `alg`, holt den Public Key des Auth-Servers (gecached über JWKS) und verifiziert die Signatur über `Header.Payload` — keine DB-Anfrage, kein Roundtrip. Daraus die drei großen Vorteile: (1) **Horizontale Skalierung** — jeder API-Server prüft lokal, ohne mit einem zentralen Session-Store zu reden, was in verteilten Architekturen (Edge, Multi-Region) operativ entscheidend ist. (2) **Microservice-Tauglich** — der Token kann zwischen internen Services weitergereicht werden, jeder Service prüft selbständig die Signatur und liest die Claims; kein zentraler Engpass. (3) **Asymmetrische Signatur (RSA/ECDSA)** — nur der Auth-Server hat den Private Key zum Ausstellen, alle anderen Services brauchen nur den Public Key zum Prüfen. Damit gibt es kein geteiltes Geheimnis, das gehütet werden müsste — der Public Key darf öffentlich liegen.

Der Preis: Widerruf vor Ablauf. Ein ausgestellter Token gilt bis `exp` — der Auth-Server kann ihn *nicht* zurückrufen, weil kein Resource Server mehr beim Auth-Server nachfragt. Stateless heißt im Kern: *kein Rückkanal*. Trotzdem braucht jedes ernsthafte System eine Möglichkeit, einen Token vor Ablauf zu entwerten — bei Logout, Passwort-Reset, Kompromittierung oder Rollenwechsel. Drei etablierte Muster:

(1) **Kurze `exp` + Refresh-Token (Standardlösung).** Der Access Token gilt 5–15 Minuten; ein langlebiges, *serverseitig invalidierbares* Refresh-Token erneuert ihn. Logout heißt dann: Refresh Token in der Auth-Server-DB löschen — der Access Token läuft binnen Minuten aus. Hybrid aus Stateless-Access (skaliert) und Stateful-Refresh (kontrollierbar).

(2) **Server-Blocklist (Denylist).** Jeder Resource Server prüft die `jti` (Token-ID) gegen eine Liste widerrufener Tokens — meist Redis mit TTL bis zur ursprünglichen `exp`. Sofortige Wirkung, aber wieder stateful und ein Roundtrip pro Request. In der Praxis als Hybrid eingesetzt: Blocklist nur für kritische Tokens (Admin-Sessions, Healthcare-Zugriffe), nicht für jedes JWT.

(3) **Key-Rotation als Notbremse.** Bei Major-Incident (Datenleck, Private-Key-Diebstahl) rotiert der Auth-Server seinen Signing Key. Alle bisher ausgestellten Tokens werden ungültig, weil ihre Signaturen mit dem neuen Key nicht mehr verifizieren. Nuklear-Option, nicht selektiv.

Sicherheits-Fallen (regelmäßig in CTFs und Bug Bounties): `alg: none` erlaubt unsignierte Tokens, manche Libraries akzeptieren das aus historischen Gründen — die Algorithmen-Erwartung muss explizit gesetzt sein. `alg`-Confusion-Attacks tauschen RSA gegen HMAC und nutzen den Public Key als HMAC-Secret. Der Payload ist nur signiert, *nicht verschlüsselt* — wer PII oder Passwort-Hashes hineinschreibt, hat sie effektiv im Klartext über Caches und Logs laufen lassen. Wer Vertraulichkeit braucht, sucht JWE.

Zusammenfassung — HTTP in der Praxis

Semantik statt Syntax

- **Methoden** kommunizieren Absicht — `GET` liest, `POST` schreibt nicht-idempotent, `PUT`/`DELETE` sind idempotent.
- **Statuscodes** sind die maschinenlesbare Antwort — `2xx` Erfolg, `3xx` Weiterleitung, `4xx` Client-Fehler, `5xx` Server-Fehler.
- **Header** steuern Format, Aushandlung, Caching, Auth und Sessions — kein Beiwerk.

Performance: Caching mit Trade-off

- `Cache-Control: max-age` reduziert Last und Latenz — Preis: zeitweise veraltete Daten.
- `ETag` + `If-None-Match` ermöglichen Validierung ohne erneute Übertragung (`304 Not Modified`).
- Cache-Ebenen wirken kumulativ: Browser, CDN/Proxy, Application-Cache.

Navigation und Schreiboperationen

- **PRG** (`POST` → `303` → `GET`) verhindert doppelte Verarbeitung bei Reload.
- **301/308** dauerhaft, **302/307** temporär; **307/308** erhalten Methode und Body — Pflicht für APIs.

Zustand trotz Zustandslosigkeit

- **Cookies** stabilisieren Sessions und werden domain-gebunden mitgesendet (Same-Origin).
- Sicherheits-Attribute orthogonal: `HttpOnly` (XSS), `SameSite` (CSRF), `Secure` (Sniffing).
- Dieselbe Mechanik ermöglicht Tracking — direkt über Third-Party-Cookies oder per ID-Sync via Redirect.

Transport: HTTPS / TLS

- Vertraulichkeit, Integrität, Authentizität — aber nur für die Leitung, nicht für die Anwendungslogik.

HTTP bleibt simpel auf Protokollebene, wird aber durch Header und Statuscodes zum ausdrucksstarken Steuerungsmechanismus. Caching ist der wichtigste Performance-Hebel — aber jede Caching-Entscheidung ist ein Trade-off zwischen Aktualität und Last. Redirects steuern Navigation, PRG verhindert Doppelverarbeitung. Cookies ermöglichen Zustand trotz Zustandslosigkeit — aber nur mit korrekter Konfiguration. TLS sichert den Transport.

Speaker notes

Diese Folie fasst das HTTP-Kapitel zusammen, bevor wir HTTP-Semantik gezielt auf API-Design anwenden und anschließend REST als systematisierende Architekturidee betrachten. Die wichtigste Botschaft: HTTP ist nicht nur Transport, sondern ein semantisches Protokoll. Methoden, Statuscodes und Header bilden gemeinsam ein Vokabular, das Infrastruktur (Browser, Caches, CDNs, Load Balancer, Monitoring) ohne Anwendungswissen verstehen kann. Wer dieses Vokabular korrekt verwendet, gewinnt Cachebarkeit, Wiederholbarkeit, Skalierbarkeit, Sicherheit und Beobachtbarkeit fast geschenkt; wer es ignoriert (z.B. immer `200 OK` mit Fehler-JSON, alles über `POST`), verliert genau diese Vorteile. Die nächsten Kapitel zeigen erst, wie diese Bausteine in API-Designs konkret eingesetzt werden, und dann, wie REST sie zu Architekturprinzipien systematisiert.

REST als Architekturidee

Leitfrage: Was unterscheidet eine RESTful API von einem beliebigen HTTP-Endpunkt, der nur JSON ausliefert?

Speaker notes

REST ist kein Protokoll, sondern ein Architekturstil für verteilte Systeme. Er baut auf Web-Grundprinzipien auf und nutzt HTTP, URIs (Uniform Resource Identifiers), Repräsentationen und Caching bewusst als Infrastrukturmechanismen.

REST kurz eingeführt

> **REST** steht für **Representational State Transfer**.

REST entstand nicht als Framework oder Produkt, sondern als Beschreibung dafuer, **wie das Web bereits erfolgreich funktioniert**:

- Ressourcen haben stabile Adressen (`/products/42`)
- Clients navigieren über standardisierte HTTP-Operationen
- Server liefern **Repräsentationen** dieser Ressourcen
- Infrastruktur wie Browser, Caches und Proxies kann mitarbeiten

Entstehungskontext

- Ende der 1990er Jahre im Umfeld der Web-Architektur
- Geprägt durch **Roy Fielding**
- Ausgangsfrage: Warum skaliert das Web trotz seiner Einfachheit so gut?



Für die Praxis: REST ist zuerst ein **Programmier- und Entwurfsparadigma für verteilte Client-Server-Systeme**. Die formale Einordnung als Architekturstil kommt im naechsten Schritt.

Speaker notes

REST (Representational State Transfer) wurde 2000 von Roy Fielding in seiner Dissertation „Architectural Styles and the Design of Network-based Software Architectures” beschrieben — nicht als neue Erfindung, sondern als Formalisierung der Prinzipien, die das Web von Anfang an skalierbar gemacht haben. Fielding war maßgeblich an der Spezifikation von HTTP/1.1 beteiligt und analysierte im Rückblick, welche Architekturentscheidungen den Erfolg des Webs erklären. Der Name betont den Kern: Übertragen wird nicht die Ressource selbst (die ist ein abstraktes Konzept auf dem Server), sondern eine Repräsentation ihres aktuellen Zustands — identifiziert über eine URI. REST ist damit primär ein Beschreibungsmodell für HTTP-Nutzung, kein eigenständiges Protokoll.

REST: Ein Architekturstil, kein Protokoll

REST ist kein einzelnes Protokoll, sondern ein Satz von Constraints für verteilte Systeme.

Constraint	Bedeutung	Konsequenz
Client-Server	Klare Rollentrennung	Client und Server entwickeln sich unabhängig
Stateless	Jeder Request enthält alle nötigen Informationen	Server skaliert horizontal, kein Session-Zustand
Cacheable	Responses deklarieren ihre Cachebarkeit	Infrastruktur kann Caching transparent umsetzen
Uniform Interface	Einheitliche Schnittstelle für alle Ressourcen	Generische Werkzeuge (Browser, curl, Proxies) funktionieren
Layered System	Client weiß nicht, ob er direkt mit dem Server spricht	Load Balancer, CDN, API Gateway transparent einfügbar
Code on Demand (optional)	Server kann ausführbaren Code liefern	JavaScript im Browser

Kernidee

> REST nutzt die **vorhandene Web-Infrastruktur** (HTTP, URIs, Caching, Proxies) als Anwendungsplattform — statt ein eigenes RPC-Protokoll darüber zu bauen.

Speaker notes

Die zentralen REST-Constraints sind Client-Server, Statelessness, Cacheability, Uniform Interface und Layered System; Code on Demand ist optional. Diese Constraints sollen Skalierbarkeit, Entkopplung und Wiederverwendbarkeit von Infrastruktur verbessern.

Welchen Constraint verletzt dieses Design?

Szenario 1: Der Online-Shop speichert den Warenkorb in einer serverseitigen Session. Beim nächsten Request muss der Client denselben Server treffen (Sticky Sessions).

Szenario 2: Die API liefert bei `GET /api/products` keinen `Cache-Control`-Header. Jeder Request geht bis zum Origin-Server.

Szenario 3: Der Client kennt die interne Datenbankstruktur und baut SQL-artige Queries: `GET /api/query?table=products&where=price>50`

Szenario	Verletzter Constraint	Konsequenz
1: Sticky Sessions	**Stateless**	Horizontale Skalierung eingeschränkt, Server-Ausfall = Session verloren
2: Kein Cache-Control	**Cacheable**	CDN nutzlos, unnötige Serverlast, höhere Latenz
3: SQL-artige Queries	**Uniform Interface** + **Layered System**	Client an Interna gekoppelt, keine Abstraktion möglich

Speaker notes

Sticky Sessions verletzen das Ideal der Statelessness, fehlende Cache-Deklaration verhindert Cacheability und ein direkter Bezug auf interne Datenstrukturen verletzt das Uniform Interface. REST bewertet APIs also nicht nur nach URI-Form, sondern nach ihrem Verhalten als verteiltes System.

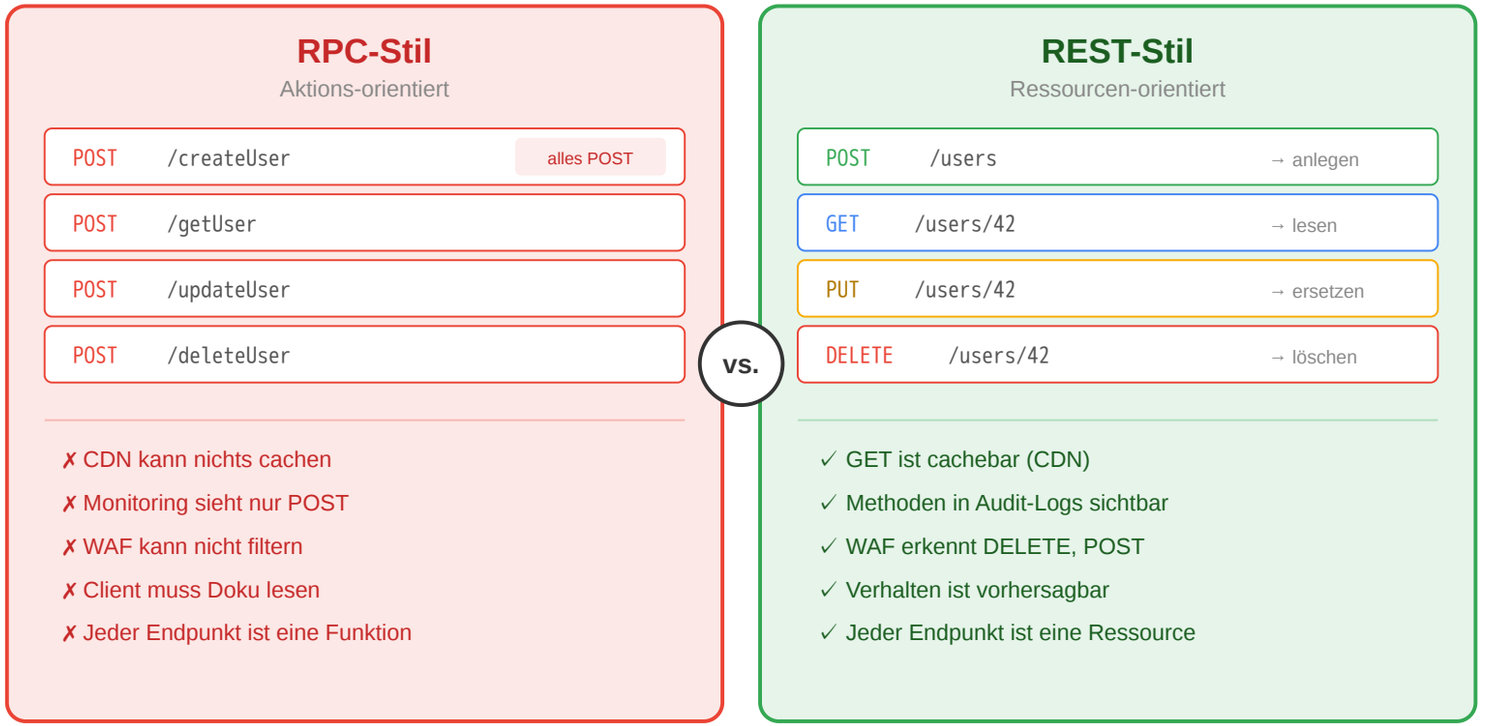
REST vs. RPC im Vergleich

RPC-Stil (aktionsorientiert)

- Jeder Endpunkt ist eine **Funktion**
- HTTP-Methode immer POST
- Infrastruktur kann nichts cachen
- Client muss Doku lesen

REST-Stil (ressourcenorientiert)

- Jeder Endpunkt ist eine **Ressource**
- HTTP-Methoden drücken Absicht aus
- GET-Requests sind cachebar
- Verhalten ist vorhersagbar



Warum das für Infrastruktur relevant ist

CDN, WAF und Monitoring verstehen HTTP-Semantik: GET wird gecacht, POST als schreibend erkannt, DELETE in Audit-Logs hervorgehoben. RPC-Stil verliert all das — jeder POST sieht gleich aus.

Speaker notes

RPC modelliert Endpunkte oft als Funktionen oder Aktionen. REST modelliert fachliche Ressourcen und nutzt HTTP-Methoden für die Operationen auf ihnen. Dadurch kann Infrastruktur HTTP-Semantik besser auswerten, etwa für Caching, Logging, Security oder Monitoring.

Ressourcen und Repräsentationen

Eine **Ressource** ist ein fachliches Konzept. Was der Client empfaengt, ist eine **Repräsentation**.

Konzept	Bedeutung
Ressource	Das fachliche Ding (existiert im System)
URI	Die Adresse der Ressource (identifiziert sie eindeutig)
Repräsentation	Die konkrete Darstellung (JSON, HTML, CSV - je nach `Accept`-Header)
Zustandsübergang	Aktion auf der Ressource (GET liest, POST erzeugt, DELETE entfernt)

> **Content Negotiation:** Client und Server einigen sich über `Accept` und `Content-Type` auf das Format.

> Dieselbe Ressource kann also über dieselbe URI identifiziert werden, während ihre konkrete Darstellung je nach Client unterschiedlich ausfaellt.

Speaker notes

Eine Ressource ist das fachliche Objekt im System, etwa ein Produkt oder eine Bestellung. Eine Repräsentation ist die konkrete Darstellung dieser Ressource, zum Beispiel JSON (JavaScript Object Notation), HTML (Hypertext Markup Language) oder CSV (Comma-Separated Values). Die URI identifiziert die Ressource; `Content-Type` und `Accept` betreffen ihre Darstellung. Genau diese Trennung erklärt auch, warum dieselbe Ressource in verschiedenen Formaten geliefert oder durch verschiedene Clients unterschiedlich verarbeitet werden kann.

Wann stößt REST an Grenzen?

Situation	REST-Problem	Alternative
Client braucht Daten aus 5 Ressourcen gleichzeitig	5 separate Requests → Latenz	**GraphQL**: Ein Query, ein Response
Echtzeit-Updates (Chat, Live-Ticker)	Polling ist ineffizient	**WebSockets**: bidirektionale Verbindung
Interne Microservice-Kommunikation	JSON-Overhead, keine Typsicherheit	**gRPC**: binäres Protokoll, Protobuf-Schema
Komplexe Aktionen (Batch-Operationen)	Passt nicht in CRUD	**RPC-Endpunkte** als Ergänzung

Kein Entweder-Oder

Viele Systeme **kombinieren** Stile: REST für öffentliche APIs, GraphQL für flexible Frontends, gRPC für interne Services. Die Wahl hängt von **Nutzungsszenarien** ab — nicht von Dogma.

> REST ist kein JSON-over-HTTP. Es ist ein Architekturstil mit sechs Constraints, die das Web skalierbar, cachebar und erweiterbar machen. Die Stärke liegt in der konsequenten Nutzung bestehender Web-Infrastruktur. Wer die Constraints versteht, kann beurteilen, wann REST die richtige Wahl ist und wann ein anderer Stil besser passt.

Speaker notes

REST passt gut für viele Web-APIs, aber nicht für alle Probleme. GraphQL eignet sich für flexible zusammengesetzte Datenabfragen, WebSockets für Echtzeitkommunikation und gRPC (Google Remote Procedure Call) für effiziente typsichere Service-zu-Service-Kommunikation. REST bleibt trotzdem oft ein robuster Default für öffentliche APIs.

Ressourcen und URI-Design

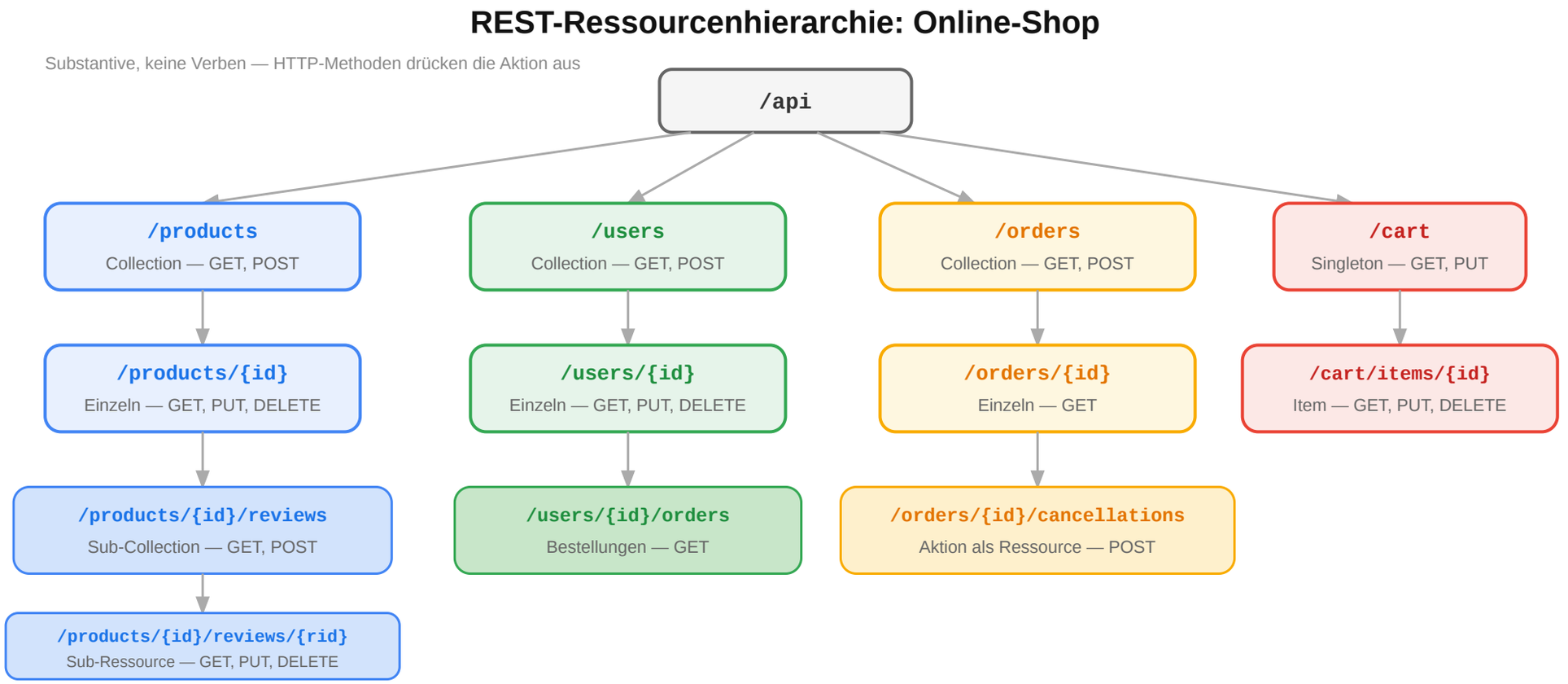
Leitfrage: Wie werden fachliche Objekte so als Ressourcen modelliert, dass eine API verständlich und stabil bleibt?

Speaker notes

Gute URI-Designs machen Ressourcen, Zugehörigkeit und Filter logisch sichtbar. Dabei sollte die URI fachliche Stabilität ausdrucken und nicht die interne Datenbank oder Implementierung spiegeln.

Ressourcenorientiertes Design: Online-Shop

Der erste Schritt beim API-Design ist die Identifikation der **Ressourcen** — die fachlichen Dinge, mit denen Clients interagieren:



Speaker notes

Typische Ressourcen eines Online-Shops sind Produkte, Kategorien, Warenkoerbe, Bestellungen und Bewertungen. Zwischen Ressourcen können Hierarchien oder Beziehungen bestehen, die sich in Collections und Unterressourcen ausdruecken lassen.

URI-Design-Regeln

Regel	Gut	Schlecht	Warum
Plural für Collections	`/products`	`/product`	Collection enthält viele
Kleinschreibung, Bindestriche	`/order-items`	`/OrderItems`	URL-Konvention
Keine Verben im Pfad	`POST /orders`	`POST /createOrder`	Methode drückt Aktion aus
Hierarchie = Zugehörigkeit	`/users/42/orders`	`/getUserOrders?userId=42`	Beziehung sichtbar
Query für Filter/Sortierung	`/products?sort=price`	`/products/sortByPrice`	Pfad = Identität, Query = Parameter

Anti-Patterns erkennen

```
POST /api/getUser             →  ?
POST /api/deleteAccount       →  ?
GET  /api/search?action=find  →  ?
POST /api/doPayment           →  ?
```

Ressourcenorientierte Fassung

```
POST /api/getUser             →  GET    /users/{id}
POST /api/deleteAccount       →  DELETE /accounts/{id}
GET  /api/search?action=find  →  GET    /products?q=...
POST /api/doPayment           →  POST   /orders/{id}/payments
```

Speaker notes

URIs sollten konsistent, lesbar und fachlich begruendet sein. Collections stehen meist im Plural, Verben gehören nach Möglichkeit in die HTTP-Methode statt in den Pfad, und Query-Parameter dienen für Filterung, Suche oder Sortierung statt für Identität.

Übung: API-Design für eine Domäne

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Speaker notes

Beim Ressourcenentwurf werden fachliche Entitäten, Beziehungen und typische Operationen identifiziert. Mehrere URI-Strukturen können valide sein, solange sie aus Sicht der API-Nutzung konsistent und verständlich sind.

Übung: API-Design für eine Domäne

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Eine mögliche Lösung

Fachliches Konzept	URI	Methoden
Alle Kurse	`/courses`	GET, POST
Ein Kurs	`/courses/web-eng`	GET, PUT, DELETE
Vorlesungen eines Kurses	`/courses/web-eng/lectures`	GET, POST
Einschreibungen	`/courses/web-eng/enrollments`	GET, POST

Fachliches Konzept	URI	Methoden
Eine Einschreibung	`/courses/web-eng/enrollments/42`	GET, PUT
Noten eines Studierenden	`/students/42/grades`	GET
Note in einem Kurs	`/students/42/grades/web-eng`	GET, PUT

> **Diskussion:** Sollte die Note unter `/students/42/grades/web-eng` oder unter `/courses/web-eng/grades/42` liegen? Beide sind valide — die Wahl hängt davon ab, **wer der primäre Nutzer der API** ist.

CRUD-Mapping auf HTTP

Operation	HTTP	URI	Body	Response
Liste lesen	`GET`	`/products`	—	`200` + Array
Einzeln lesen	`GET`	`/products/42`	—	`200` + Objekt
Anlegen	`POST`	`/products`	Neues Objekt	`201` + Location
Vollständig ersetzen	`PUT`	`/products/42`	Vollständig	`200` / `204`
Teilweise ändern	`PATCH`	`/products/42`	Nur Änderungen	`200` / `204`
Löschen	`DELETE`	`/products/42`	—	`204`

Jenseits von CRUD

Nicht alles passt in CRUD. Für **Aktionen** gibt es zwei Muster:

- **Sub-Ressource:** `POST /orders/42/cancellations`
- **Controller-Ressource:** `POST /password-resets`

Speaker notes

CRUD (Create, Read, Update, Delete) deckt den häufigsten Fall — datenzentrierte Ressourcen — gut ab, stößt aber bei aktionsorientierten Operationen an Grenzen. Eine Bestellung stornieren ist keine `DELETE`-Operation auf der Bestellung selbst (die Bestellung bleibt erhalten, ihr Status ändert sich). Zwei etablierte Muster für solche Fälle: **Sub-Ressource** (`POST /orders/42/cancellations` — die Stornierung ist eine eigenständige Ressource, die durch `POST` entsteht) und **Controller-Ressource** (`POST /password-resets` — eine Ressource, die eine Aktion kapselt). Beide Muster bleiben HTTP-konform und vermeiden RPC-artige Verb-URLs wie `/cancelOrder?id=42`.

Workflow: Create / Update / Delete korrekt abbilden

Typische Zustandsänderungen und passende Responses

POST /products
201 Created + Location

PUT /products/42
200 OK oder 204

PATCH /products/42
200 OK oder 204

DELETE /products/42
204 No Content

POST erzeugt typischerweise eine neue Ressource, PUT/PATCH ändern eine vorhandene, DELETE entfernt sie.
Der Statuscode soll den fachlichen Effekt ausdrücken, nicht nur „irgendwie Erfolg“.

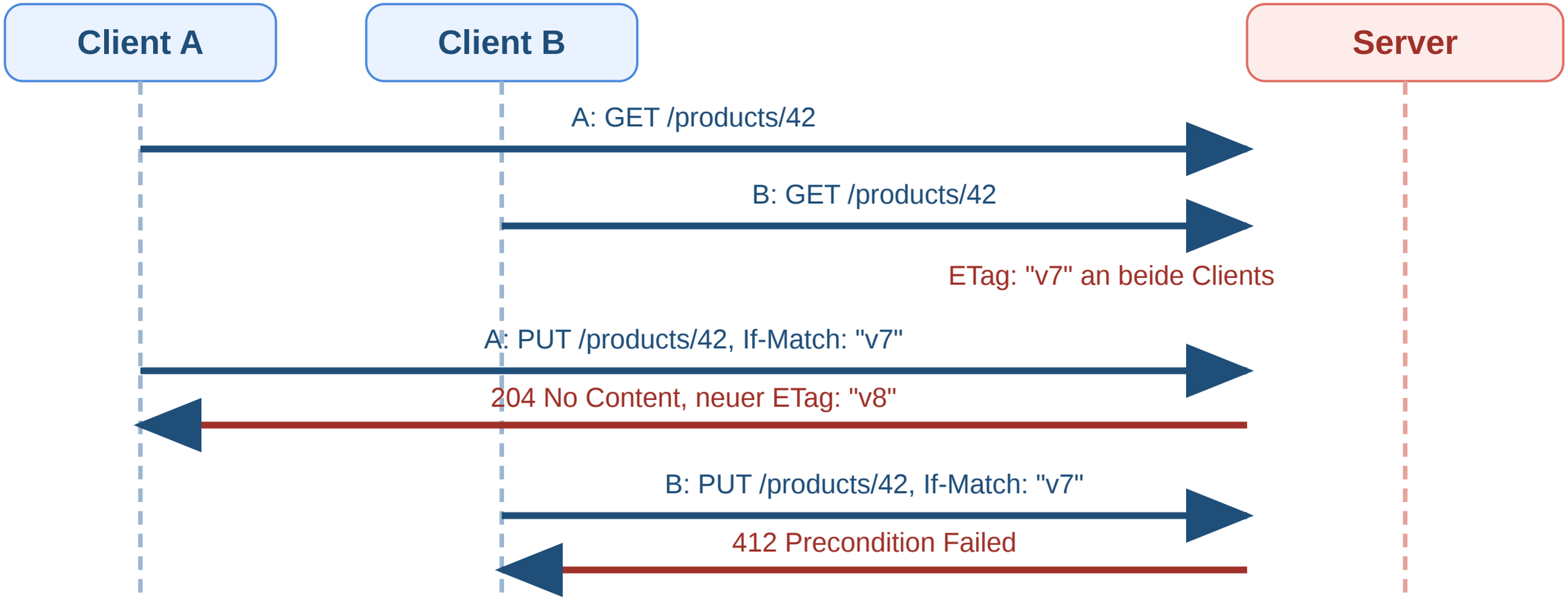
Operation	Gute Response	Warum?
Anlegen	`201 Created` + `Location`	Neue Ressource ist entstanden
Aktualisieren	`200` oder `204`	Ressource existierte bereits
Löschen	`204 No Content`	Erfolgreich, aber kein Body nötig

Speaker notes

Dieser Workflow zeigt das typische Grundmuster für zustandsaendernde Ressourcenoperationen. Wichtig ist die Unterscheidung zwischen Erzeugen (`201 Created`) und Veraendern oder Löschen (`200` oder `204`). Die `Location`-Header-Angabe ist besonders bei neu erstellten Ressourcen wertvoll. MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/201 und https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/204

Workflow: Conditional PUT mit ETag und If-Match

Ziel: Verhindern von konfligierenden Updates. Das PUT wird nur ausgeführt, wenn die Ressource noch die erwartete Version hat.



Speaker notes

Das ist der typische Conditional-PUT-Workflow. Der Client sendet die neue Repräsentation nur unter der Bedingung, dass der aktuelle `ETag` noch zum zuletzt gelesenen Zustand passt. So wird verhindert, dass eine inzwischen veraltete Clientkopie neuere Änderungen überschreibt. `If-Match` formuliert diese Vorbedingung; bei Konflikt ist `412 Precondition Failed` passend. MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/If-Match und https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/412

Pagination, Filter und Sortierung

Query-Parameter beschreiben Auswahl, Reihenfolge und Ausschnitt einer Collection — nicht die Identität der Ressource selbst.

```
GET /products?page=2&per_page=20&sort=-price&category=electronics&q=kopfhörer
```

Parameter	Funktion	Beispiel
page / per_page	Seitenbasierte Pagination	?page=3&per_page=25
cursor	Cursor-basierte Pagination	?cursor=eyJpZCI6NDJ9
sort	Sortierung (- = absteigend)	?sort=-created_at,name
filter	Filterung	?status=active&min_price=10
q	Volltextsuche	?q=kopfhörer

Pagination in der Response

```
{
  "data": [{"id": 42, "name": "Funkkopfhörer", "price": 79.99}],
  "meta": { "total": 142, "page": 2, "per_page": 20 },
  "links": {
    "next": "/products?page=3&per_page=20",
    "prev": "/products?page=1&per_page=20"
  }
}
```

Links machen die API navigierbar — der Client muss keine URIs konstruieren.

Gute REST-APIs beginnen mit sauber geschnittenen Ressourcen. URI-Design ist eine **Modellierungsaufgabe**: Ressourcen identifizieren, Beziehungen als Hierarchie abbilden, Aktionen über

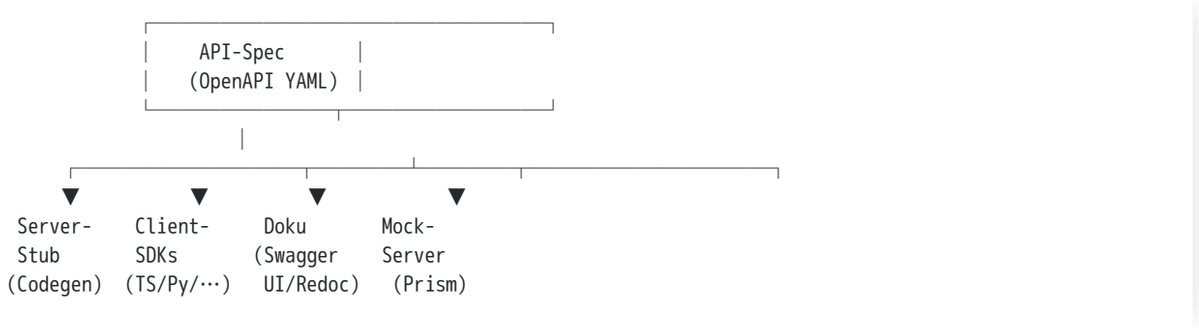
HTTP-Methoden ausdrücken. Die Entscheidung, wo eine Ressource in der Hierarchie steht, hängt vom Nutzungskontext ab — nicht von der Datenbankstruktur.

Speaker notes

Collection-Ressourcen brauchen oft zusätzliche Abfrageparameter. Pagination begrenzt die Menge der Daten, Sortierung ordnet sie, Filter schraenken sie fachlich ein. Response-Metadaten und Links helfen Clients, weitere Seiten oder Suchergebnisse systematisch zu navigieren.

Die API als zentrales Design-Artefakt

Idee: Die API-Spezifikation ist nicht Beiwerk zur Implementierung, sondern das **zentrale Design-Artefakt**. Aus ihr werden alle anderen Bausteine synchronisiert.



Vorteil: Eine Quelle hält alles synchron. Spec-Änderung → Doku-Update, SDK-Update, neue Mock-Antworten, aktualisierte Test-Erwartungen.

API-First als Strategie — das Bezos-Mandat (Amazon, 2002):

> *„All teams will henceforth expose their data and functionality through service interfaces. […] There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. […] All service interfaces, without exception, must be designed from the ground up to be **externalizable**."*

— Bezos-Memo, ~2002, öffentlich rekonstruiert durch Steve Yegges Plattform-Rant (2011).

> **Konsequenz:** Jedes Amazon-Team musste sich behandeln, als wäre es ein externer Anbieter. Diese Disziplin machte aus internen Diensten später **AWS** — die Infrastruktur war API-strukturiert und damit nach außen verkaufbar.

API-Design ist also nicht nur eine technische, sondern eine **organisatorische** Entscheidung. Spec-first zwingt Teams, Schnittstellen explizit zu klären, bevor Code entsteht.

Speaker notes

Die Idee der Spec als zentrales Design-Artefakt ist der entscheidende Mindset-Shift gegenüber „Doku am Ende": Wenn alle Konsumenten der API (Frontend-Team, Mobile-Team, externe Partner, interne andere Microservices) gegen denselben generierten Mock entwickeln und denselben generierten Client benutzen, kann die API nicht mehr unbemerkt auseinanderdriften. Spec-Änderung in einem PR → Pipeline regeneriert Clients, lintet, vergleicht mit der letzten released Version, baut neue Doku — Konsumenten sehen Änderungen vor ihrem Inkrafttreten und können reagieren. Das Bezos-Mandat von 2002 ist die historisch wichtigste Anekdote dazu: Amazon erzwang per Dekret, dass alle internen Systeme strikt nur über Service-Interfaces kommunizieren dürfen, und dass diese Interfaces von Anfang an *externalizable* gestaltet werden müssen. Genau diese Disziplin — jede interne Funktionalität als API mit klarem Vertrag — schuf die Voraussetzung dafür, dass Amazon Web Services überhaupt entstehen konnte: ein Großteil dessen, was später als EC2, S3, SQS, RDS verkauft wurde, waren intern bereits saubere APIs, deren Externalisierung primär ein Frage von Billing und Auth war, nicht von Architektur. Die heute übliche Bezeichnung „API-First" beschreibt dieselbe Idee mit dem Werkzeug-Vorteil moderner Spec-Formate (OpenAPI, AsyncAPI, gRPC) und der Tool-Chain drumherum.

API-Dokumentation mit OpenAPI

Beispiel — OpenAPI 3 Spec für `GET /products/{id}`:

```yaml
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
paths:
  /products/{id}:
    get:
      summary: Produkt abrufen
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Produkt gefunden
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
        '404':
          description: Produkt nicht gefunden
```

Was die Spec definiert:

- **Endpunkte und Methoden** — `paths` listet alle URIs mit den jeweils erlaubten HTTP-Verben.
- **Parameter** — Pfad-, Query-, Header- und Cookie-Parameter mit Typ und ob `required`.
- **Request-/Response-Schemata** — Body-Strukturen via JSON Schema (`$ref` referenziert wiederverwendbare Komponenten unter `components/schemas`).
- **Statuscodes** — pro Antwort: welche Codes treten auf, mit welchem Body-Schema.

Was OpenAPI dadurch ermöglicht:

Vorteil	Erklärung
Maschinenlesbar	Codegen für Client-SDKs und Server-Stubs
Interaktive Docs	Swagger UI / Redoc rendert direkt aus YAML, mit „Try it out"
Contract Testing	Spec als Soll-Vorgabe für Property-/Konformitätstests
Single Source of Truth	Doku und Implementierung können nicht mehr auseinanderlaufen

> **OpenAPI ≠ Swagger.** Swagger ist die ursprüngliche Toolchain (heute Quasi-Synonym für die UI); der offene Standard heißt seit 2016 **OpenAPI Specification (OAS)** und wird von der Linux Foundation gepflegt.

Speaker notes

OpenAPI ist heute der De-facto-Standard für die Spezifikation von HTTP-APIs. Die YAML-Datei beschreibt Endpunkte, Parameter, Request- und Response-Schemata vollständig maschinenlesbar — Schemata folgen JSON Schema, was sie mit anderen Toolchains kompatibel macht. Eine vollständige Spec für eine mittelgroße API kann mehrere tausend Zeilen YAML haben; sie wird daher meist modular gepflegt (`$ref` auf separate Dateien) und im Repo versioniert. Die im Beispiel gezeigte Struktur ist der absolute Kern — produktive Specs ergänzen `security`-Schemen (Bearer, API Key, OAuth-Flows), `examples` für Doku und Tests, `headers` für Response-Metadaten, `tags` für Gruppierung in der Doku und `servers` für Umgebungen (dev/staging/prod). Die strikte Trennung von OpenAPI-Spec (offener Standard) und Swagger (Toolchain) ist begrifflich wichtig: die Spec gehört keinem Anbieter, die Tools sind austauschbar.

OpenAPI-Tooling im Überblick

Die wichtigsten Werkzeuge

Werkzeug	Zweck
Swagger UI / **Redoc** / **Stoplight Elements**	Interaktive HTML-Doku aus der Spec; „Try it out"-Funktion ruft echte Endpunkte auf
openapi-generator / **Kiota** (Microsoft)	Code-Generierung: Client-SDKs (TypeScript, Python, Java, …) und Server-Stubs aus der Spec
Spectral	Linter — prüft Spec gegen Style-Regeln (Naming, Pflicht-Statuscodes, Beschreibungen, Versionspräfix, …)
Postman / **Bruno** / **Hurl**	API-Tests aus der Spec, sowohl manuell als auch in CI
Prism (Stoplight)	Mock-Server, der die Spec direkt als laufende Fake-API ausführt — Frontend kann gegen die Spec entwickeln, bevor das Backend existiert
schemathesis	Property-based Fuzz-Tests: generiert Requests aus der Spec und prüft Konformität der Responses

Best Practices für den Spec-Workflow

- **Contract-first** statt Code-first: Spec wird zuerst geschrieben und im Team reviewt, dann Code dazu. Vermeidet, dass interne Implementierungsdetails in die öffentliche API durchschlagen.
- **Spec lebt im Repo**, versioniert mit dem Code. Nicht in einem separaten Doku-Tool, das auseinanderlaufen kann.
- **Lint in CI**: Spectral als Pflicht-Check vor jedem Merge. Naming-Konsistenz, fehlende Beschreibungen, Pflicht-Statuscodes (`400`, `401`, `404`, `500`) erkennt der Linter automatisch.
- **Breaking-Change-Detection** in CI (z.B. `oasdiff`, `openapi-diff`): jeder PR wird gegen die letzte released Version verglichen; Breaking Changes erfordern Major-Versionssprung.
- **Mock-Server für Parallel-Entwicklung**: Frontend-Team kann mit Prism gegen die Spec entwickeln, während das Backend-Team implementiert.
- **Spec-driven Tests**: schemathesis o.ä. prüft im CI, dass die laufende API den Versprechen der Spec entspricht.

> Die Spec ist nicht *Doku der API*, sondern *Vertrag zur API*. Wer sie als generierten Output behandelt, verschenkt 80 % des Nutzens.

Speaker notes

Das Spec-zentrierte Tooling rund um OpenAPI hat sich in den letzten Jahren stark professionalisiert. Der entscheidende Mindset-Shift ist *contract-first*: Die OpenAPI-Spec wird zuerst geschrieben — handgepflegt im Repo, geprüft im Code-Review — und alles andere (Server-Stubs, Client-SDKs, Doku, Mocks, Tests) wird daraus generiert. Code-first-Workflows, bei denen Annotations im Code die Spec erzeugen, sind beliebt für interne Tools, machen aber öffentliche APIs schwerer evolvierbar, weil Implementierungsdetails ungewollt durchschlagen. Spectral als Linter ist heute Quasi-Standard und enthält Rule-Sets für gängige Anforderungen (alle Endpoints brauchen `description`, `operationId`, `responses` für mindestens `200` und `400`/`401`/`404`, …). Breaking-Change-Detection mit Tools wie `oasdiff` automatisiert die schwierige Frage, ob ein PR die API für Bestands-Clients bricht — ohne das müsste das jeder Reviewer manuell prüfen. Wer eine wirklich öffentliche API betreibt (externe Entwickler, SDKs in mehreren Sprachen), kommt um dieses Tooling kaum herum; für interne APIs ist es Overkill — dort genügt oft eine handgepflegte README plus ein Test-Setup.

Wrap-Up

HTTP ist ein semantisches Protokoll, kein Datentransport. Methoden drücken Absicht aus, Statuscodes kommunizieren Ergebnisse, Header steuern Caching, Sessions und Sicherheit. Infrastruktur — Browser, Caches, CDNs, Load Balancer, Monitoring — reagiert auf diese Semantik ohne Anwendungswissen.

Was wir durchgegangen sind:

- **Headers + Status Codes:** Vokabular für Metadaten und Ergebnisse — `Content-Type`, `Authorization`, `2xx`/`3xx`/`4xx`/`5xx`.
- **Representations + Caching:** Eine Ressource, viele Repräsentationen; Content Negotiation; mehrstufiges Caching mit `Cache-Control` und `ETag`; Redirects und asynchrone Workflows.
- **Sessions + CORS:** Zustand trotz Zustandslosigkeit via Cookies; Same-Origin als Isolation; CORS als kontrollierte Lockerung für moderne SPAs.
- **Authentifizierung + Sicherheit:** TLS für den Transport; API Key / Session Cookie / OAuth 2.0 / JWT für unterschiedliche Auth-Szenarien.
- **REST:** Constraints (Statelessness, Cacheability, Uniform Interface) als Architekturidee, nicht als Marketing-Begriff.
- **Ressourcen + URI-Design + OpenAPI:** fachlich stabile Ressourcen, konsistentes Verb-Mapping, Spec-zentriertes Tooling.

Die wichtigste Botschaft: Wer HTTP-Semantik korrekt nutzt, gewinnt Cachebarkeit, Skalierbarkeit, Sicherheit und Beobachtbarkeit fast geschenkt. Wer alles in `POST` mit `200 OK` und „Fehler-JSON im Body" packt, verliert genau diese Vorteile — und merkt es erst, wenn die Infrastruktur nicht mehr mitspielt.

HTTP ist das **Interaktionsmodell des Webs**. REST nutzt dieses Modell konsequent für API-Design — Ressourcen statt Funktionsaufrufe, einheitliche Schnittstelle statt proprietärer Protokolle, Evolvierbarkeit durch Spec und Versionierung statt durch Hoffnung.

Wie es weitergeht: Die nächste Vorlesung zeigt, wie diese APIs **serverseitig in Java implementiert** werden — vom HTTP-Handler bis zur Spring-Boot-Anwendung mit Persistenz und Authentifizierung.

Speaker notes

Diese Vorlesung hat HTTP als semantisches Protokoll und REST als darauf aufbauenden Architekturstil systematisch durchgenommen. Der rote Faden war durchgehend: HTTP ist mehr als Datentransport — es ist ein gemeinsames Vokabular, auf das sich Infrastruktur und Anwendungen verlassen können, sofern es korrekt genutzt wird. Die nächste Vorlesung wechselt die Perspektive von der Protokoll- zur Implementierungsebene und zeigt, wie diese Konzepte in Java mit Spring Boot konkret umgesetzt werden: HTTP-Handler, Routing, Serialisierung, Persistenz mit JPA, Authentifizierung mit Spring Security. Wer diese Vorlesung verstanden hat, wird dort weniger über das *Was* nachdenken müssen und sich auf das *Wie* konzentrieren können.